INDEX

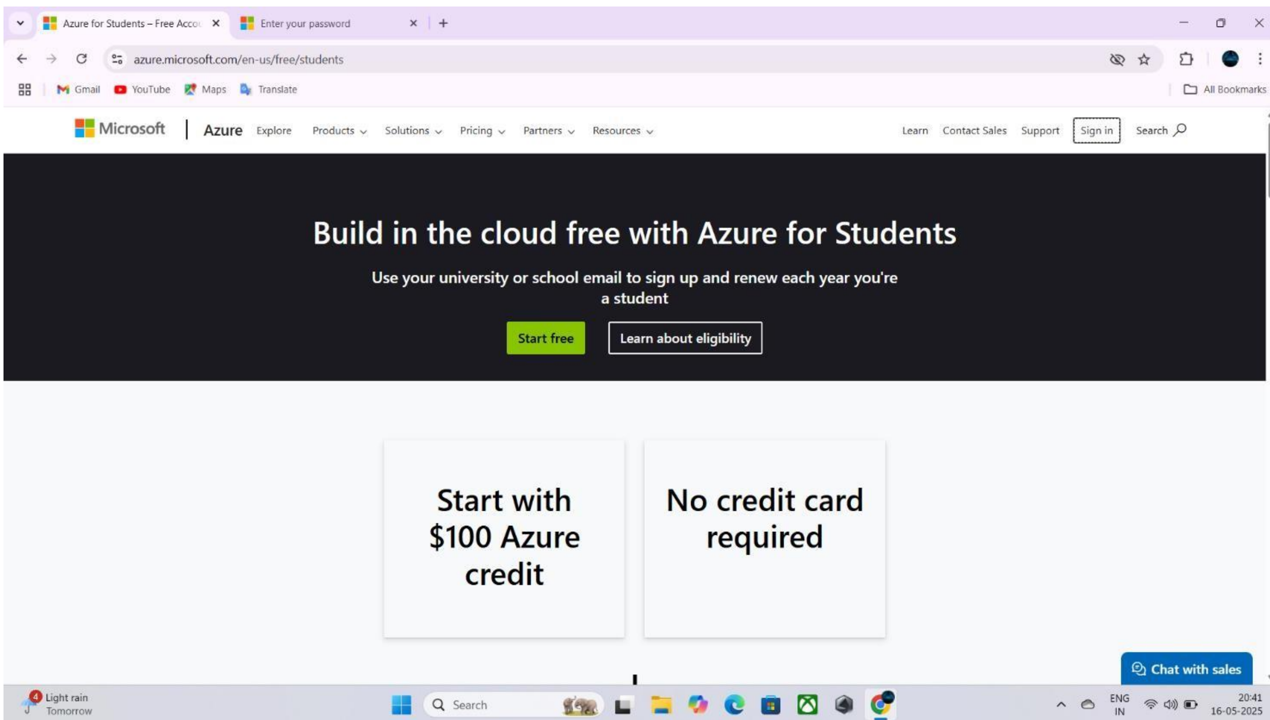
EXP NO: 1 **12/01/2026**	**AZURE DEVOPS ENVIRONMENT SETUP**

AIM:-

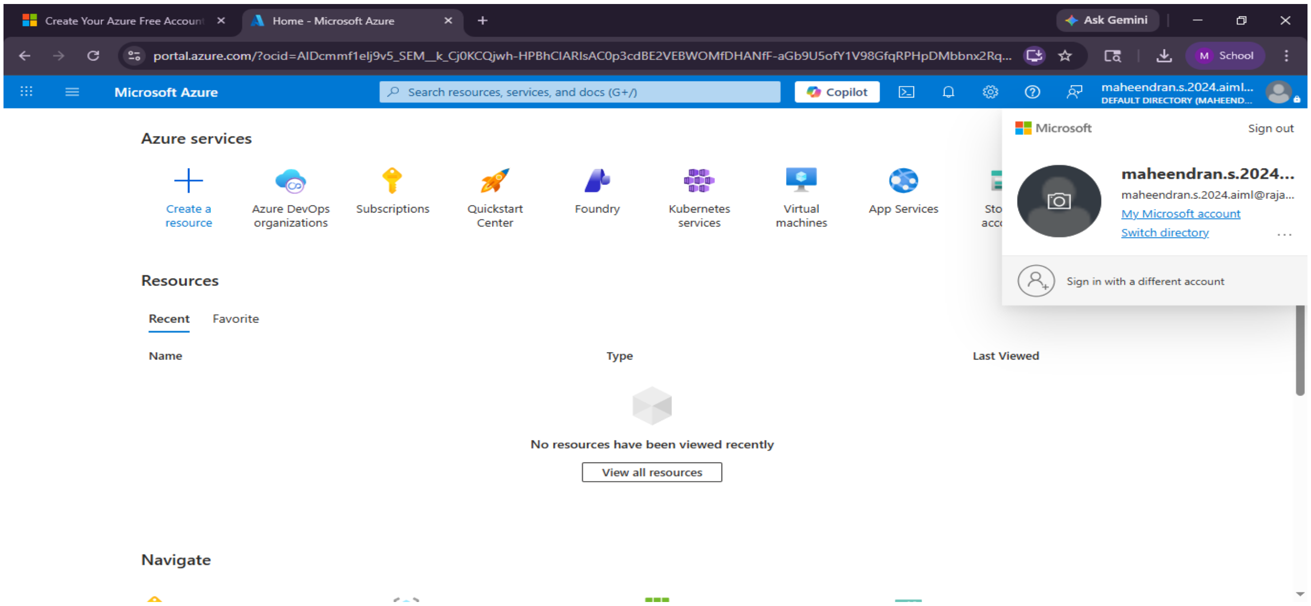
To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

INSTALLATION:-

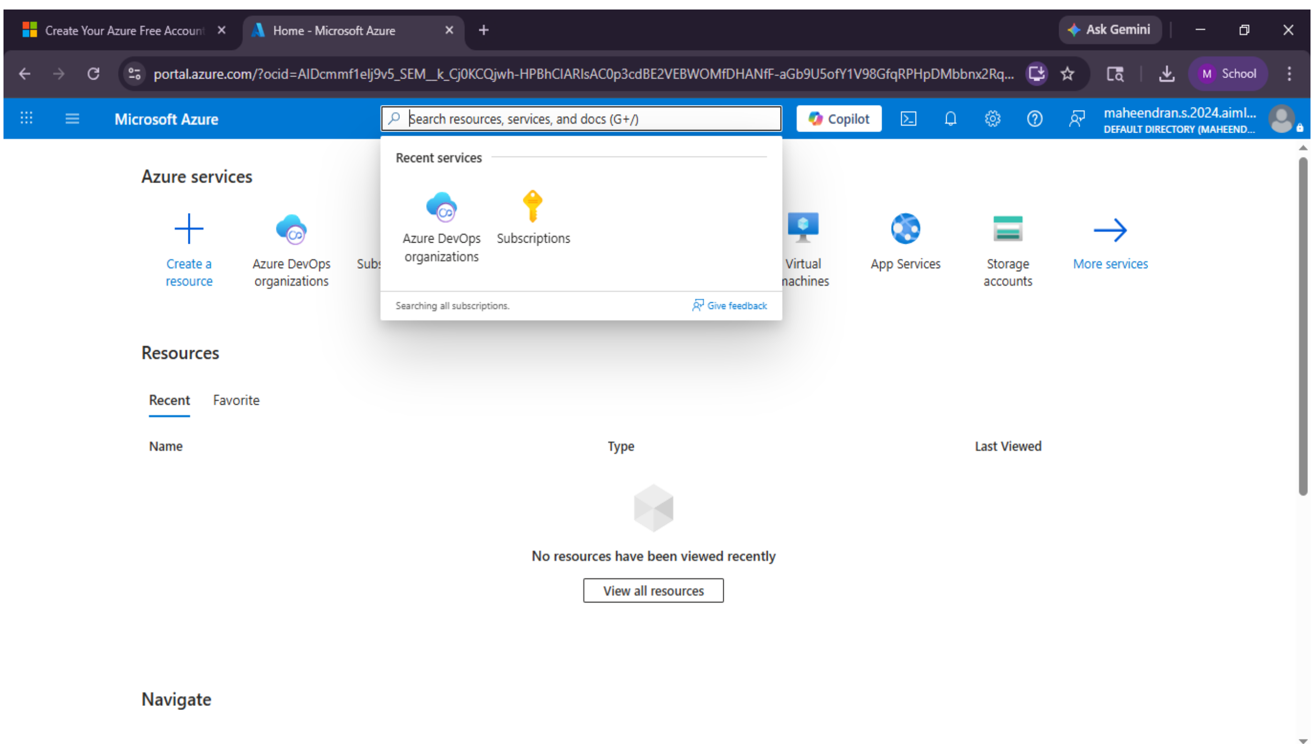
1. Open your web browser and go to the Azure website: https://azure.microsoft.com/en-us/get- started/azure-portal.
Sign in using your Microsoft account credentials.
If you don't have a Microsoft account, you can create one here: https://signup.live.com/?lic=1



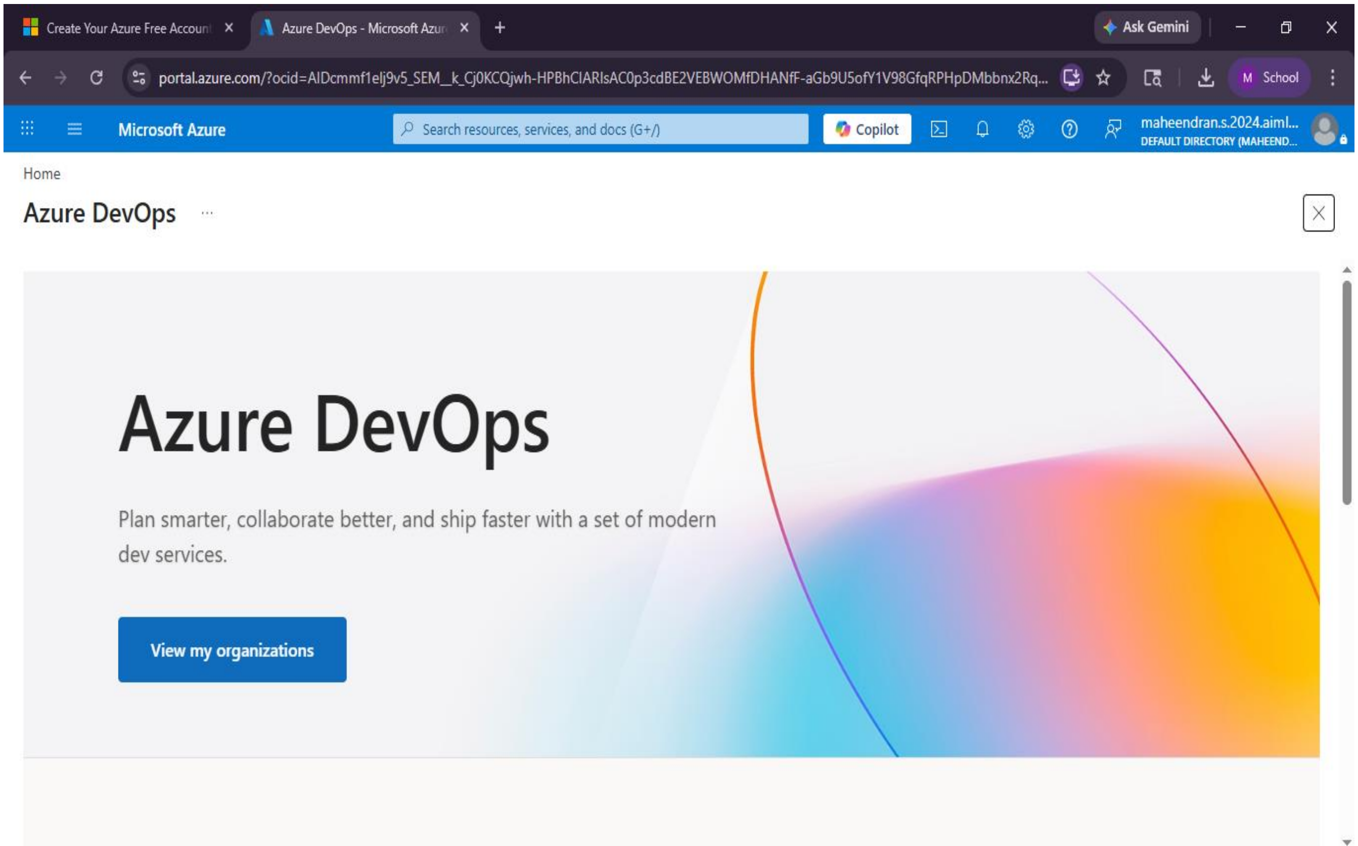
2. Azure home page



3. Open DevOps environment in the Azure platform by typing ***Azure DevOps Organizations*** in the search bar.



4. Click on the ***My Azure DevOps Organization*** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



RESULT:

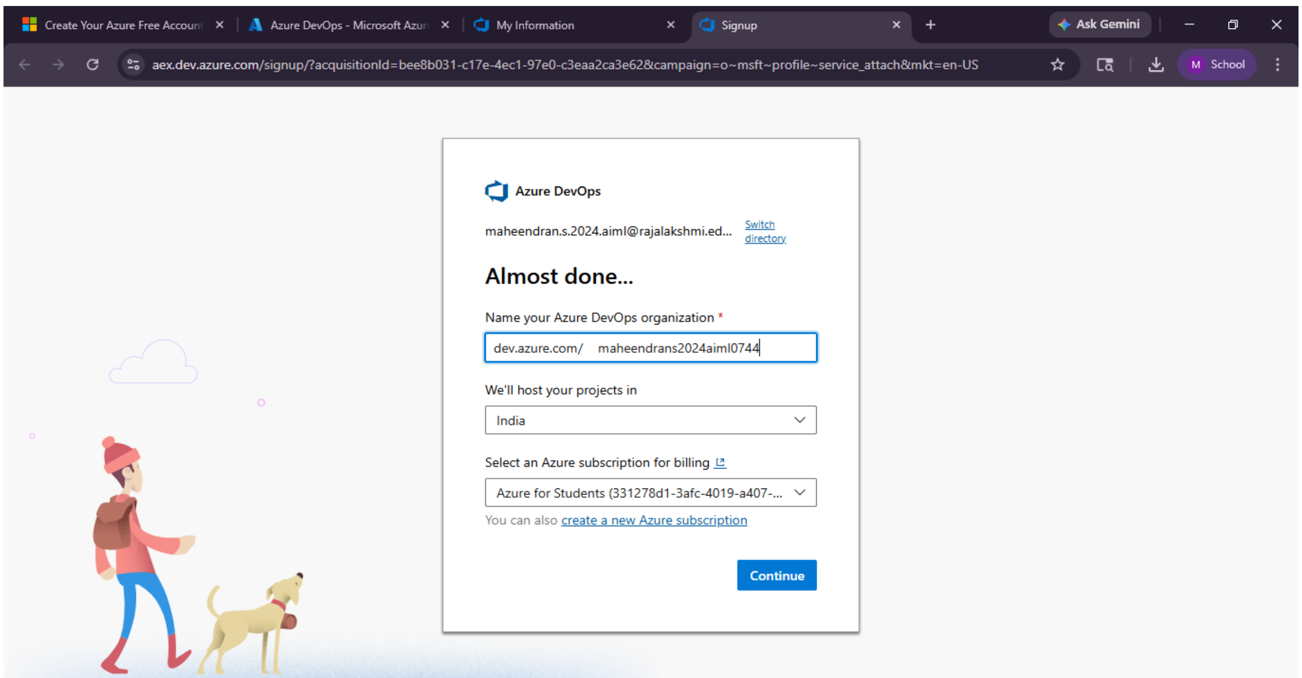
Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

EXP NO: 2 **12/01/2026**	**AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT**

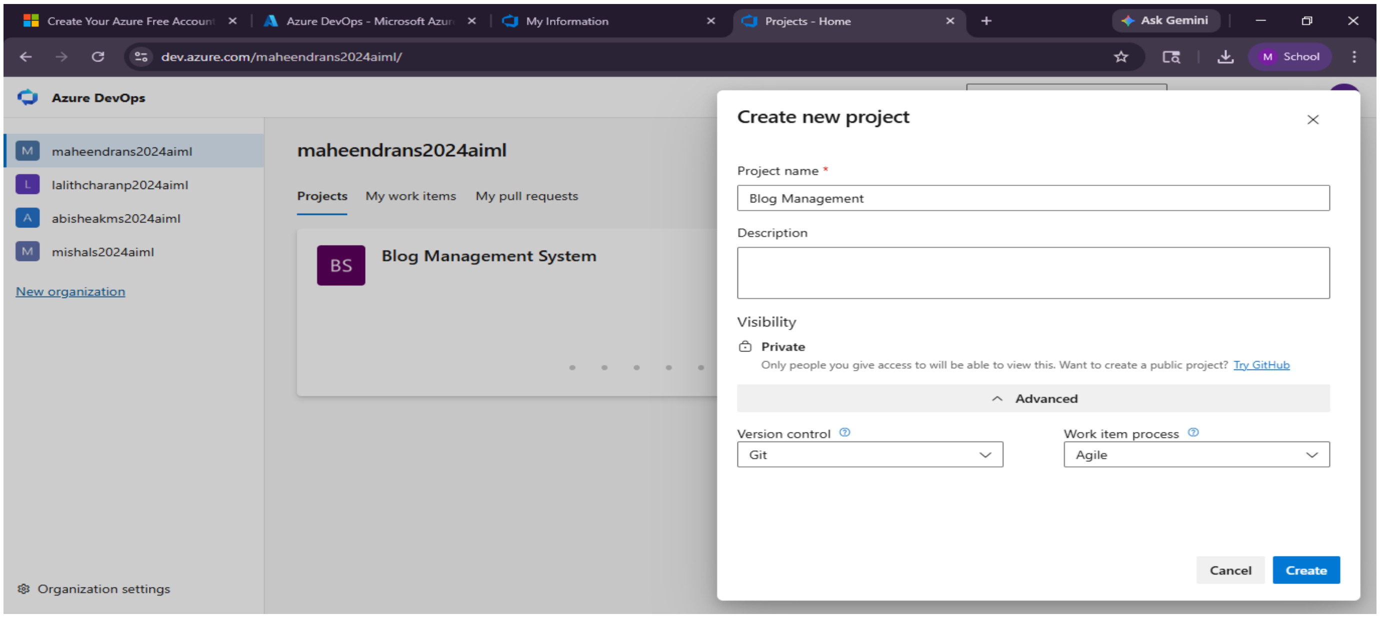
AIM:-

To set up an Azure DevOps project for efficient collaboration and agile work management.

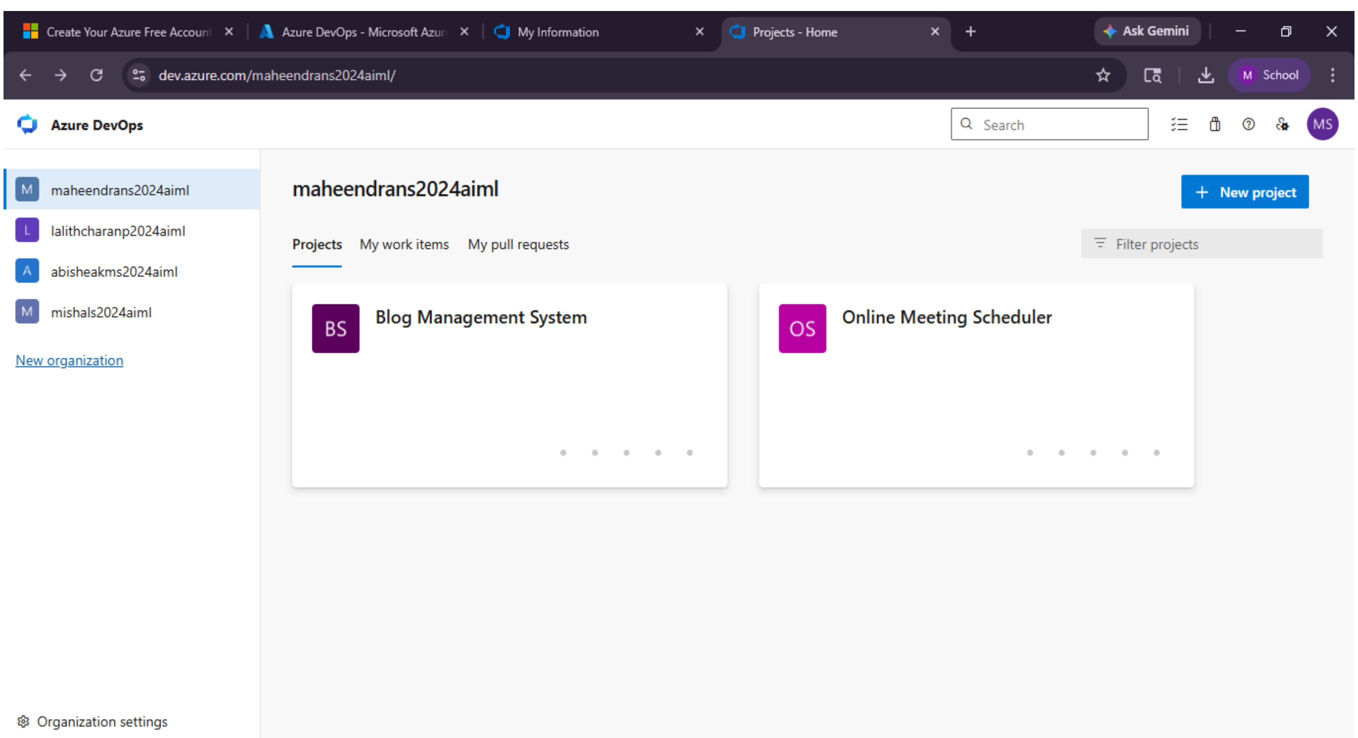
1. Create An Azure Account



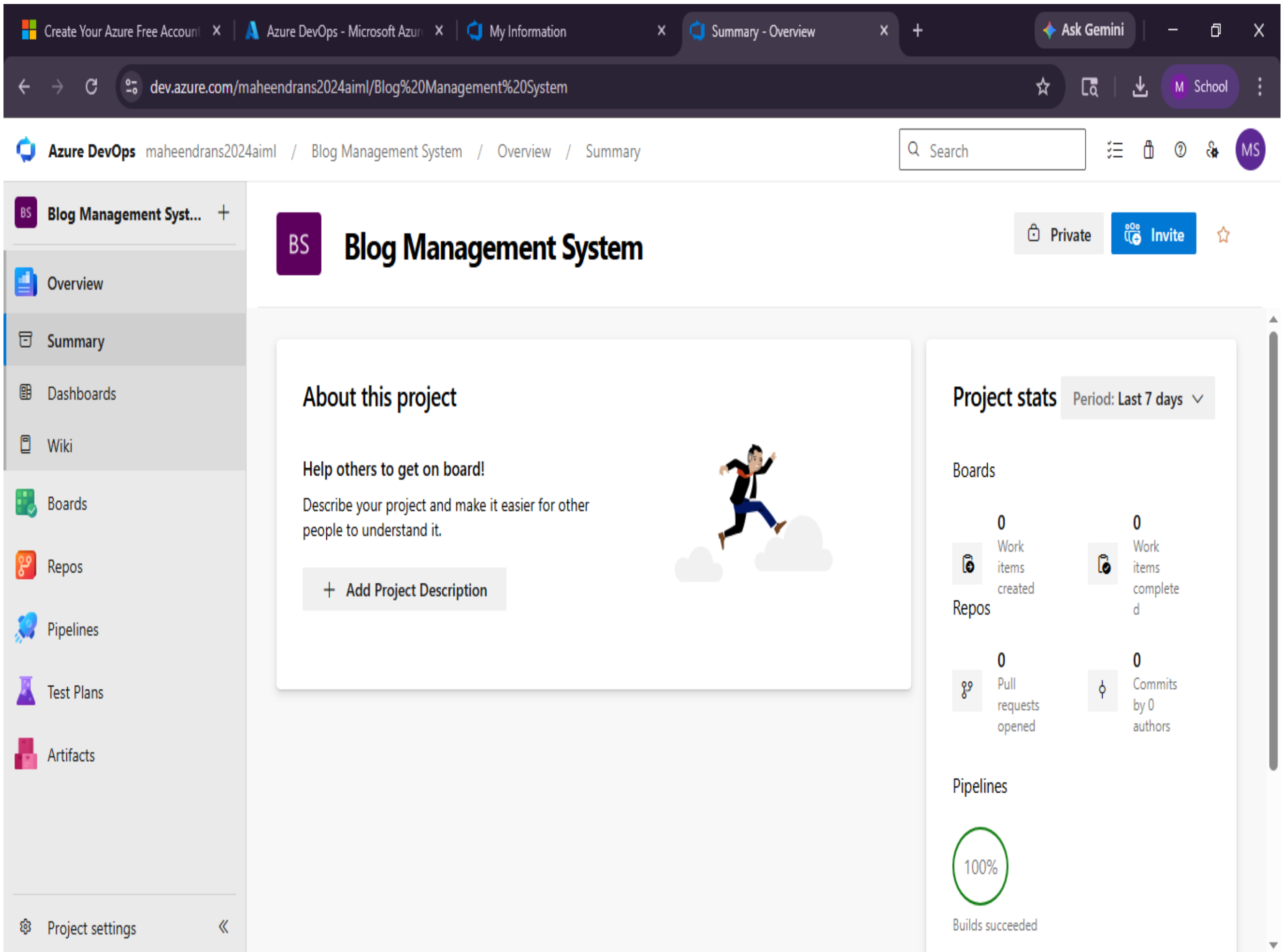
2. Create the First Project in Your Organization
   a) After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.
   b) On the organization's **Home page**, click on the **New Project** button.
   c) Enter the project name, description, and visibility options:
      Name: Choose a name for the project (e.g., LMS).
      Description: Optionally, add a description to provide more context about the project.
      Visibility: Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).
   d) Once you've filled out the details, click **Create** to set up your first project.



3. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.



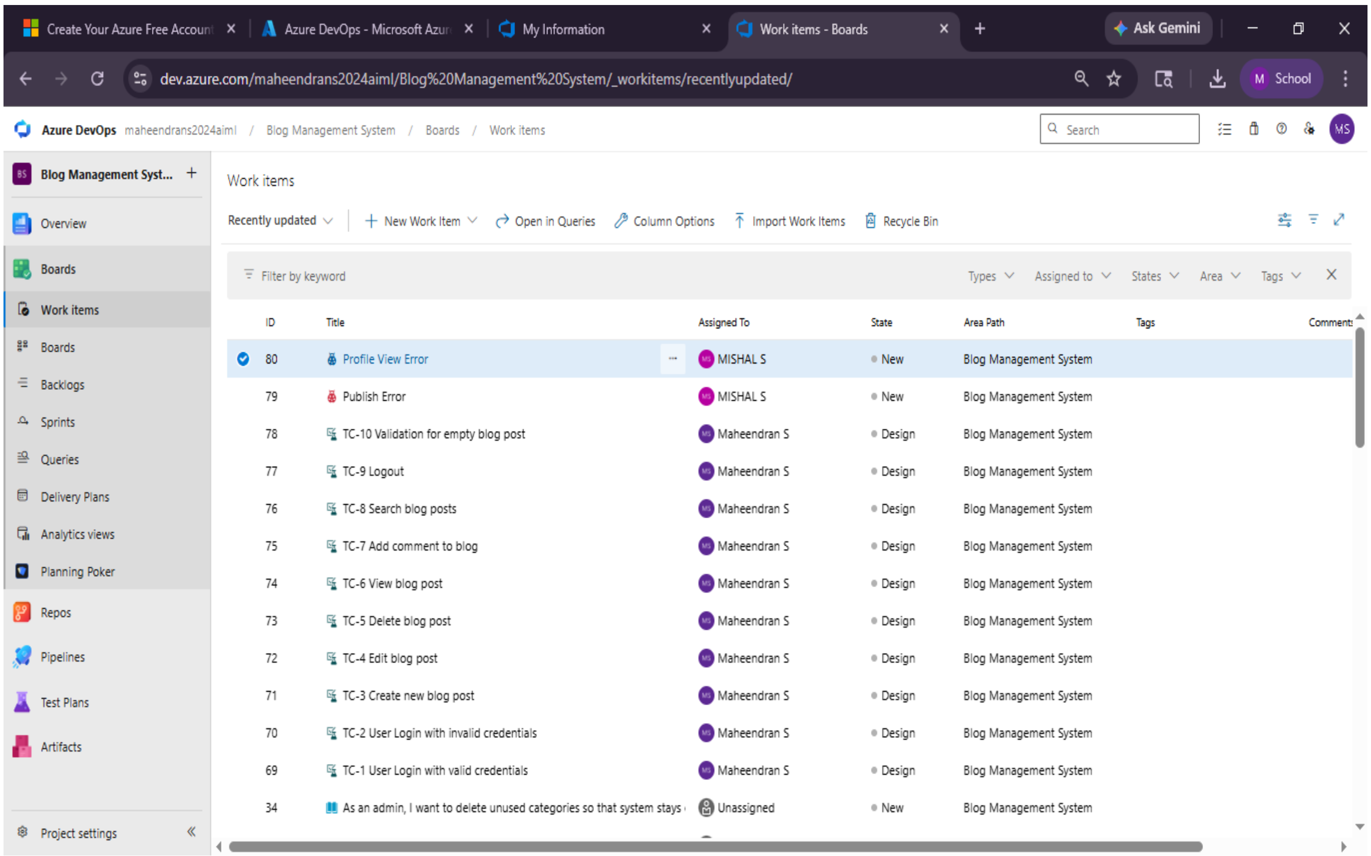
4. Project dashboard

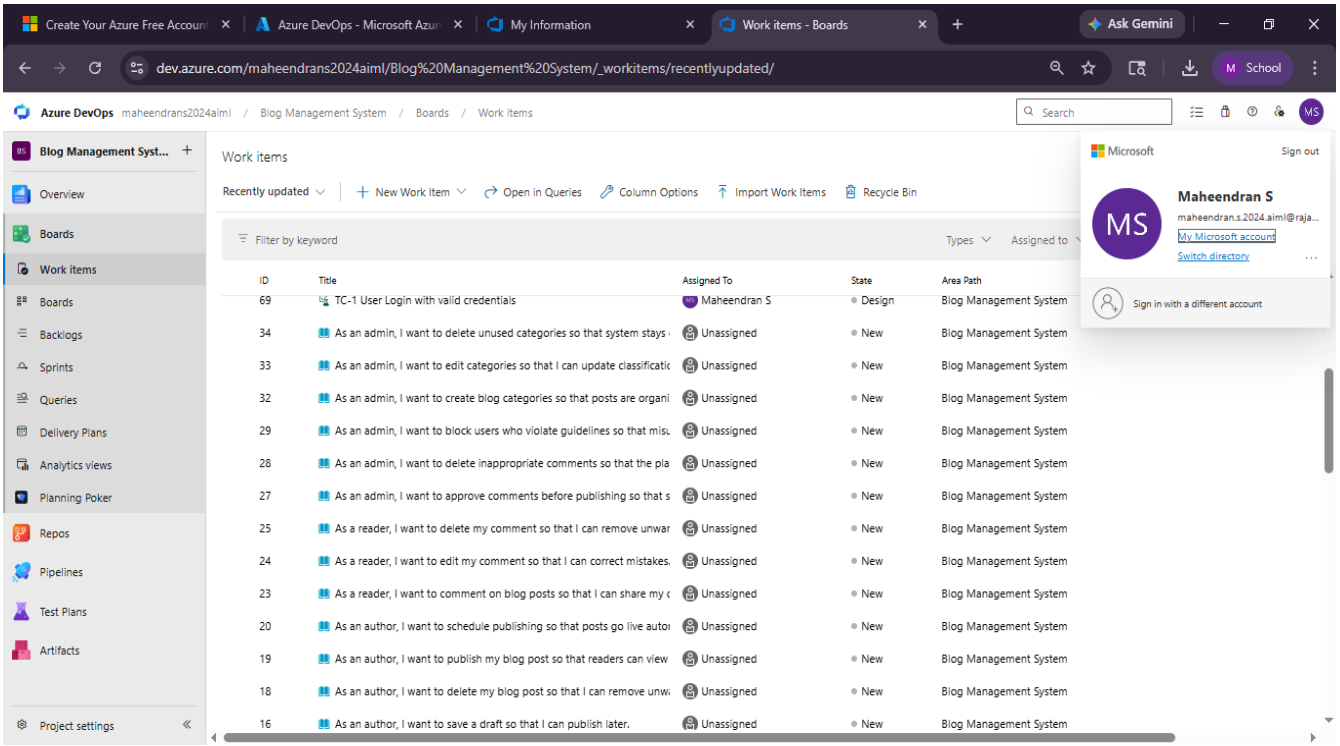


5. To manage user stories:

a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.

b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a **+** button or **Add New Work Item** depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.





Result

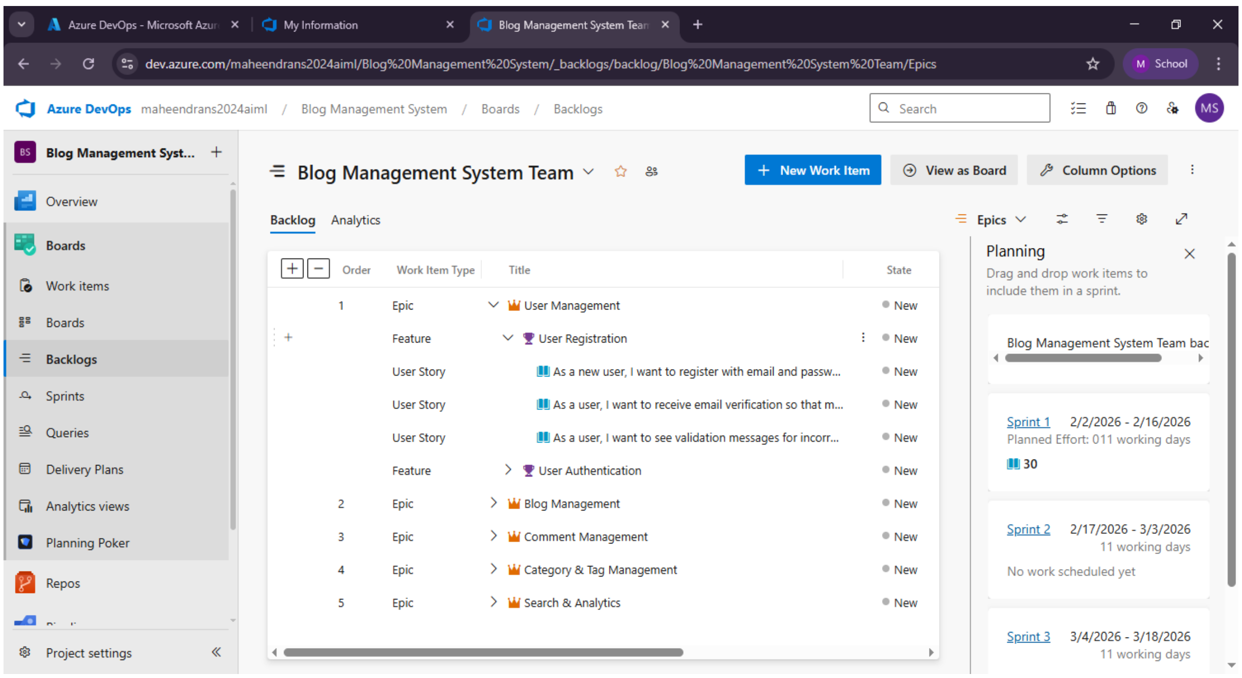
Successfully created an Azure DevOps project with user story management and agile workflow setup.

EXP NO: 3 **19/01/2026**	**SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING**

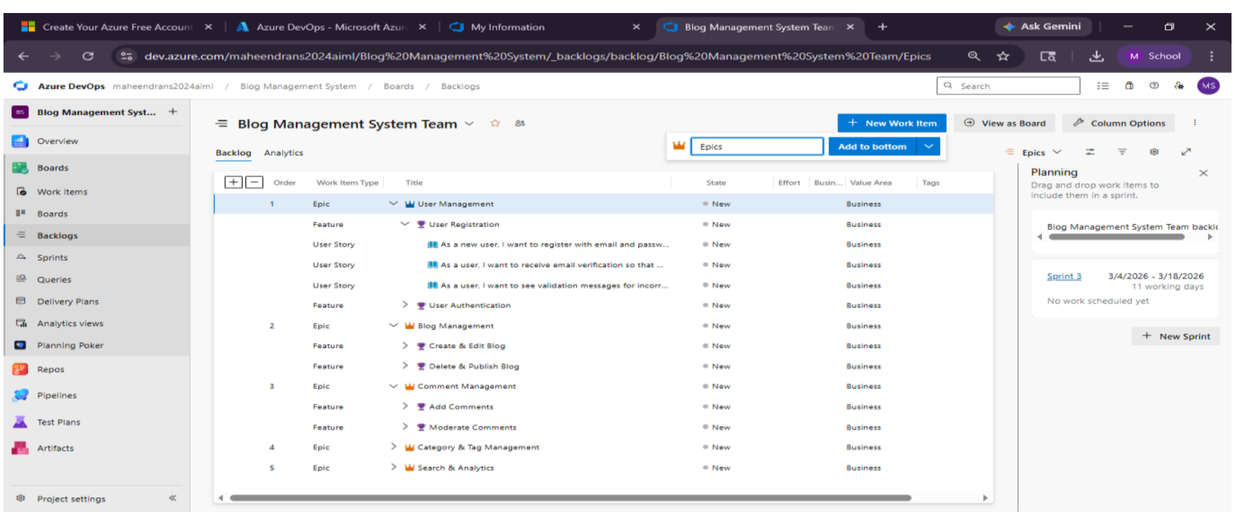
AIM:-

To learn about how to create epics, user story, features, backlogs for your assigned project.

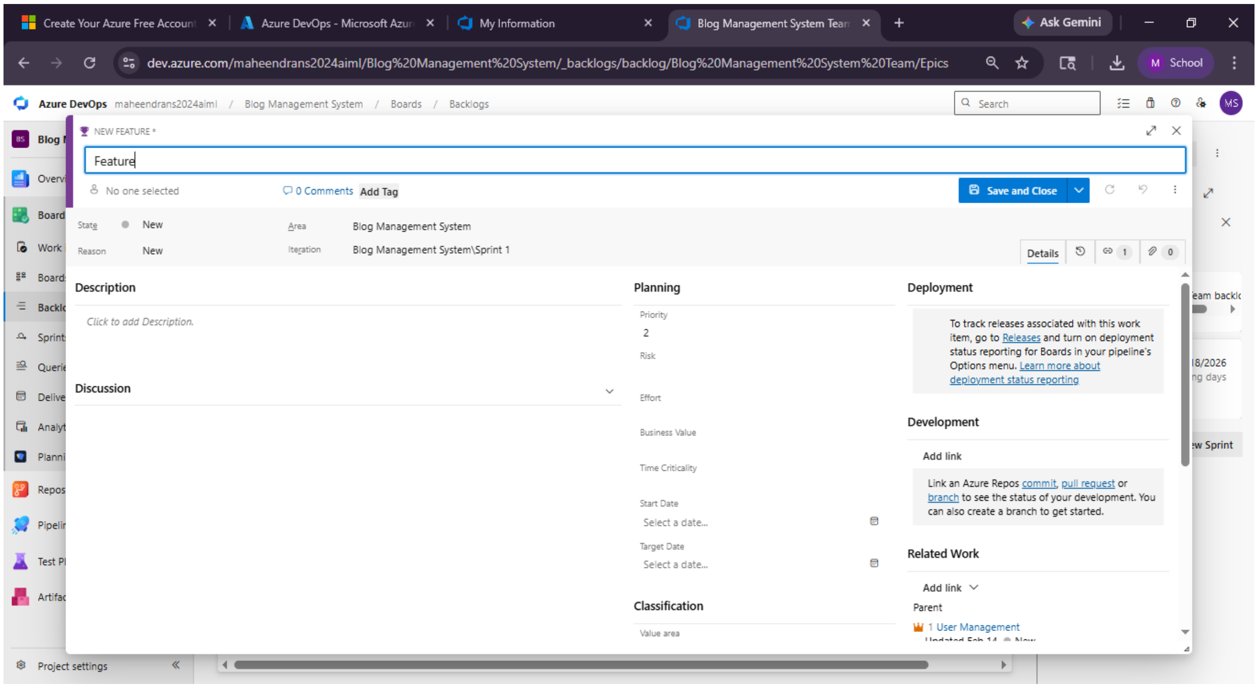
Create Epic, Features, User Stories, Task



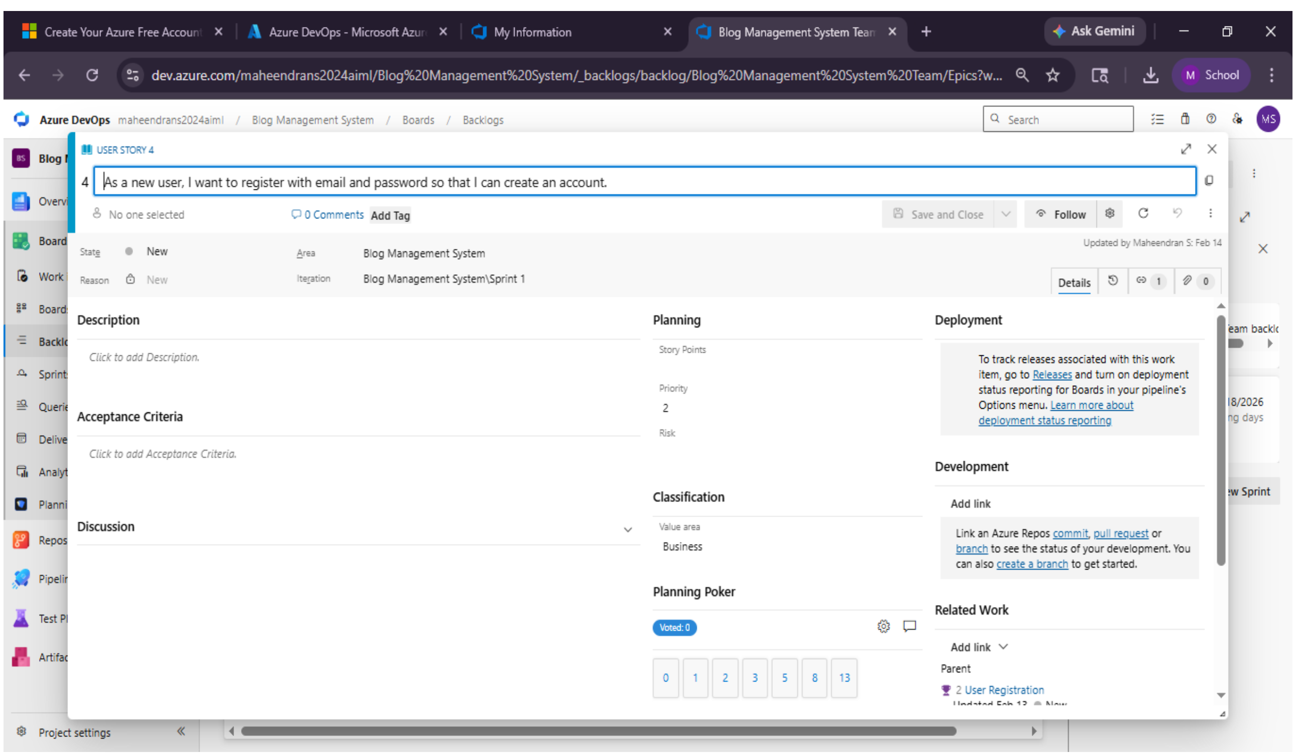
1. Fill in Epics



2. Fill in Features



3. Fill in User Story Details



RESULT:-

Thus, the creation of epics, features, user story and task has been created successfully.

EXP NO: 4 **19/01/2026**	**LIST OF FUNCTIONAL AND NON FUNCTIONAL REQUIREMENTS**

Aim

To design and develop a **Blog Management System** that allows users to create, edit, publish, and manage blog posts efficiently, while providing administrators control over content, users, and system operations.

Algorithm

1. **Start**
2. User opens the system
3. Display homepage with available blogs
4. If user selects **Register**
    - Input user details
    - Validate data
    - Store in database
5. If user selects **Login**
    - Enter username & password
    - Validate credentials
    - Grant access
6. After login:
    - Display dashboard
7. User actions:
    - Create new blog → Input content → Save → Publish
    - Edit blog → Update → Save
    - Delete blog → Confirm → Remove
8. Admin actions (if admin login):
    - Manage users
    - Approve/reject blogs
    - Monitor system activity
9. Display updated blog list
10. Logout
11. **End**

Functional Requirements

1. The system shall allow users to register by providing necessary details.
2. The system shall allow registered users to log in using valid credentials.
3. The system shall allow users to create new blog posts.
4. The system shall allow users to edit their existing blog posts.
5. The system shall allow users to delete their blog posts.
6. The system shall allow users to view all published blog posts.
7. The system shall allow users to search and filter blog posts.
8. The system shall allow users to add comments to blog posts.

Non-Functional Requirements

1. **Performance** – System should load pages within 2–3 seconds
2. **Scalability** – Should handle increasing users and blog data
3. **Security** – Protect user data with authentication and encryption
4. **Usability** – User-friendly interface for all age groups
5. **Reliability** – System should be available with minimal downtime
6. **Maintainability –** Easy to update and fix issues
7. **Compatibility** – Works across browsers and devices
8. **Backup & Recovery –** Regular data backup and restore mechanisms

Result

The Blog Management System is successfully developed to allow users to create and manage blogs efficiently. It provides a secure, scalable, and user-friendly platform for content publishing and administration, fulfilling all functional and non-functional requirements.

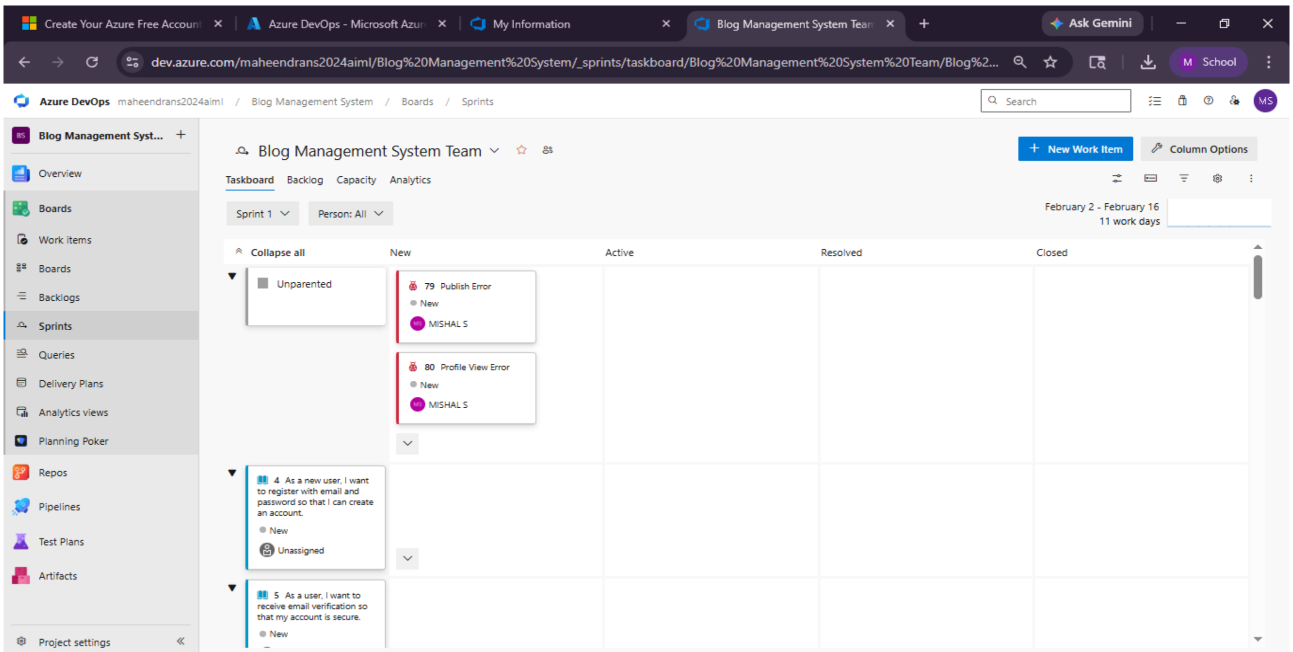
EXP NO: 5 02/02/2026	SPRINT PLANNING

AIM:-

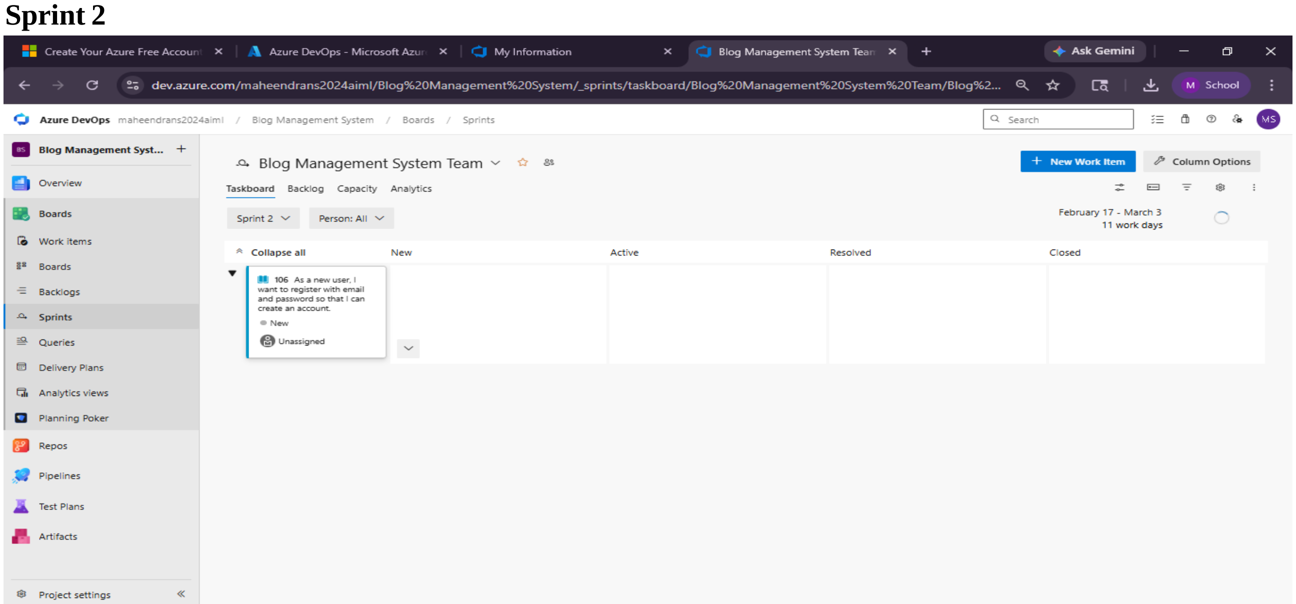
To assign user story to specific sprint for the Blog Management System Project.

Sprint Planning:-

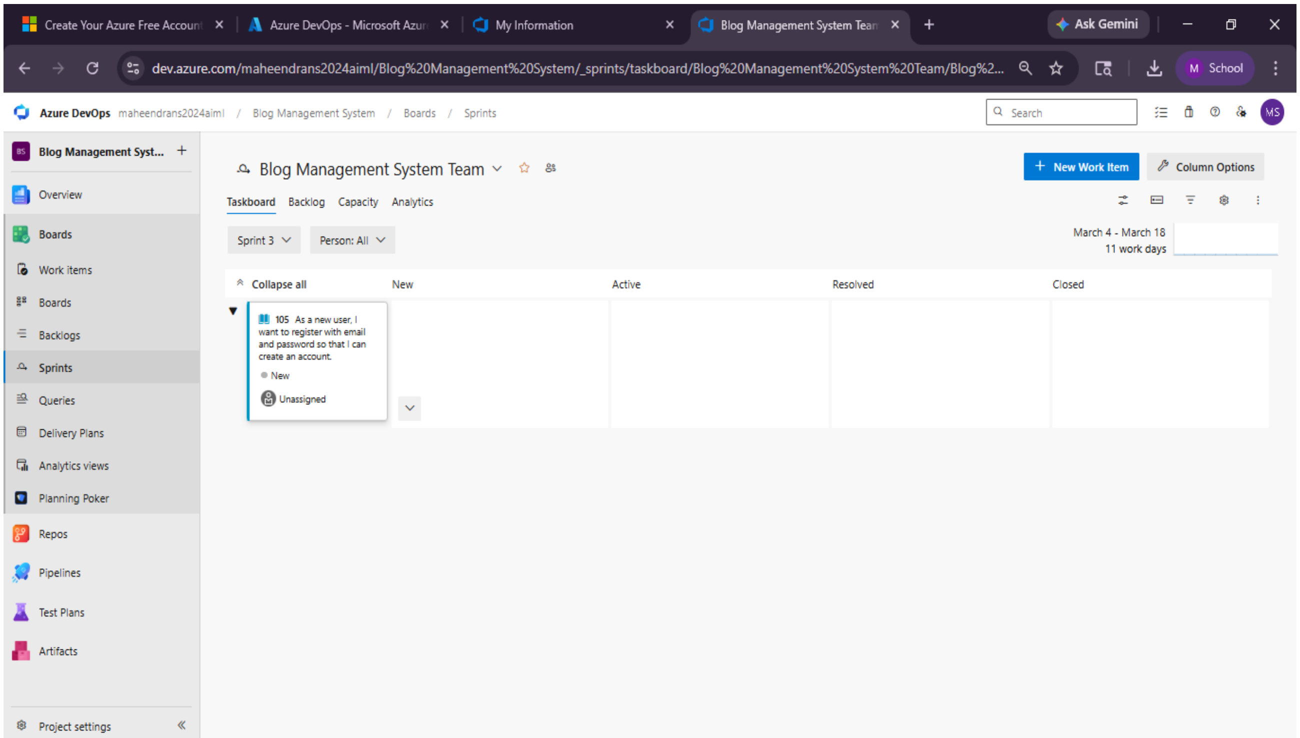
Sprint 1



Sprint 2



Sprint 3



<u>RESULT:-</u>

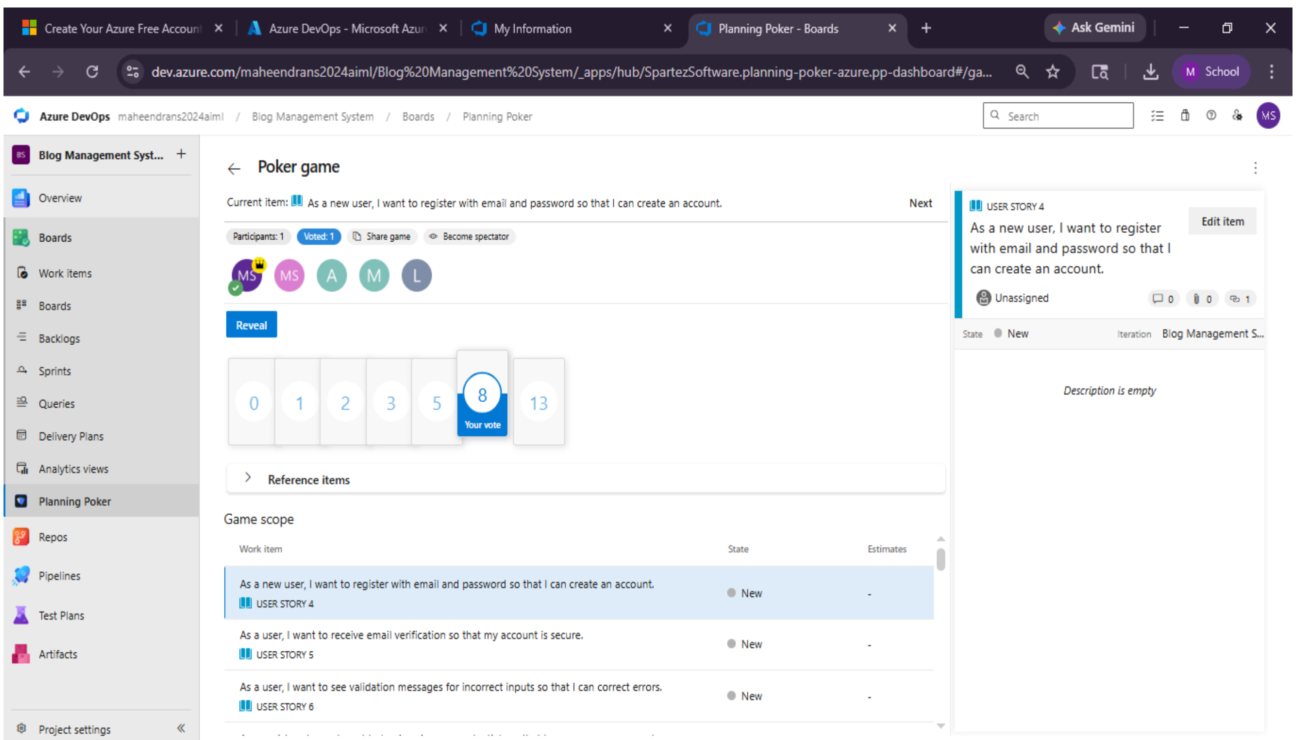
The Sprints are created for the Student Management System.

EXP NO: 6 09/02/2026	POKER ESTIMATION

AIM:-

Create Poker Estimation for the user stories -Student management System.

Poker Estimation:-



RESULT:-

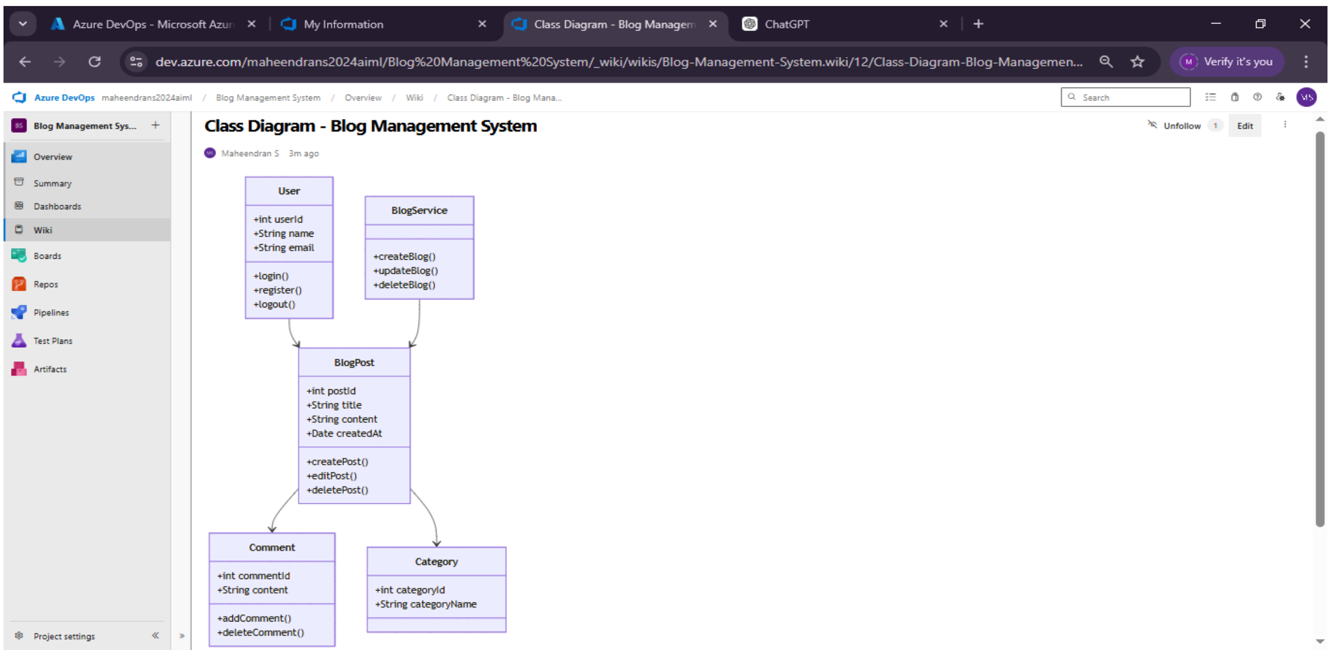
The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 7 **16/02/2026**	**DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE**

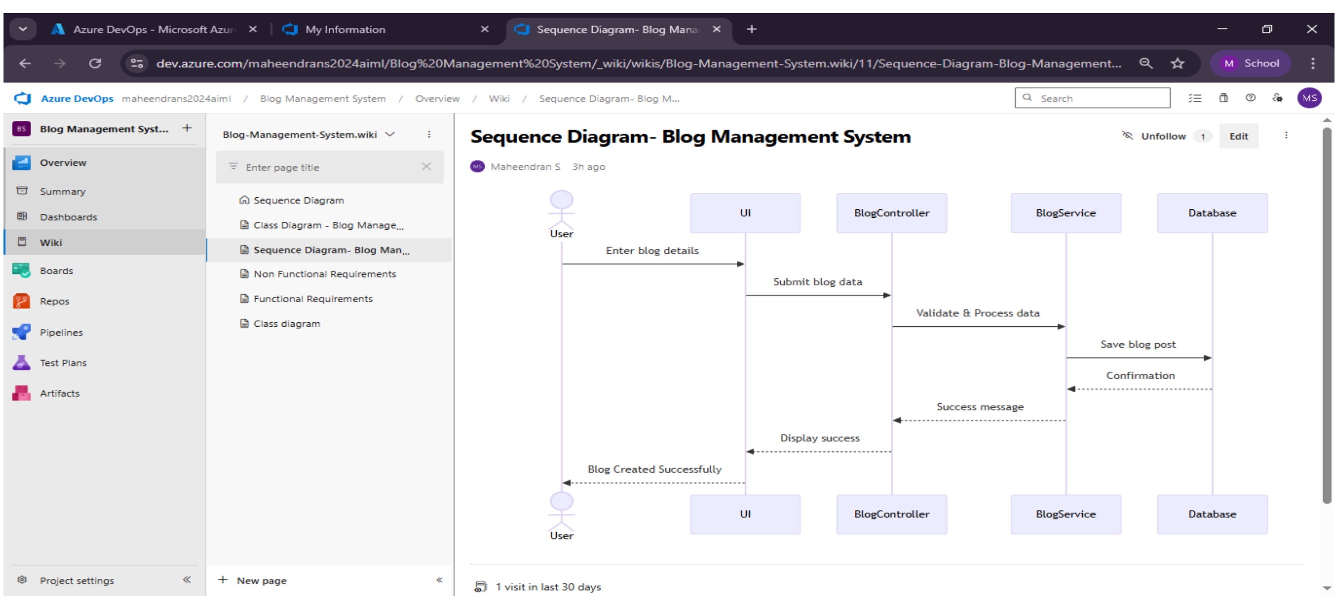
AIM:-

To Design a Class Diagram and Sequence Diagram for the given Project.

6A. CLASS DIAGRAM:-



6B. SEQUENCE DIAGRAM:-



Result:

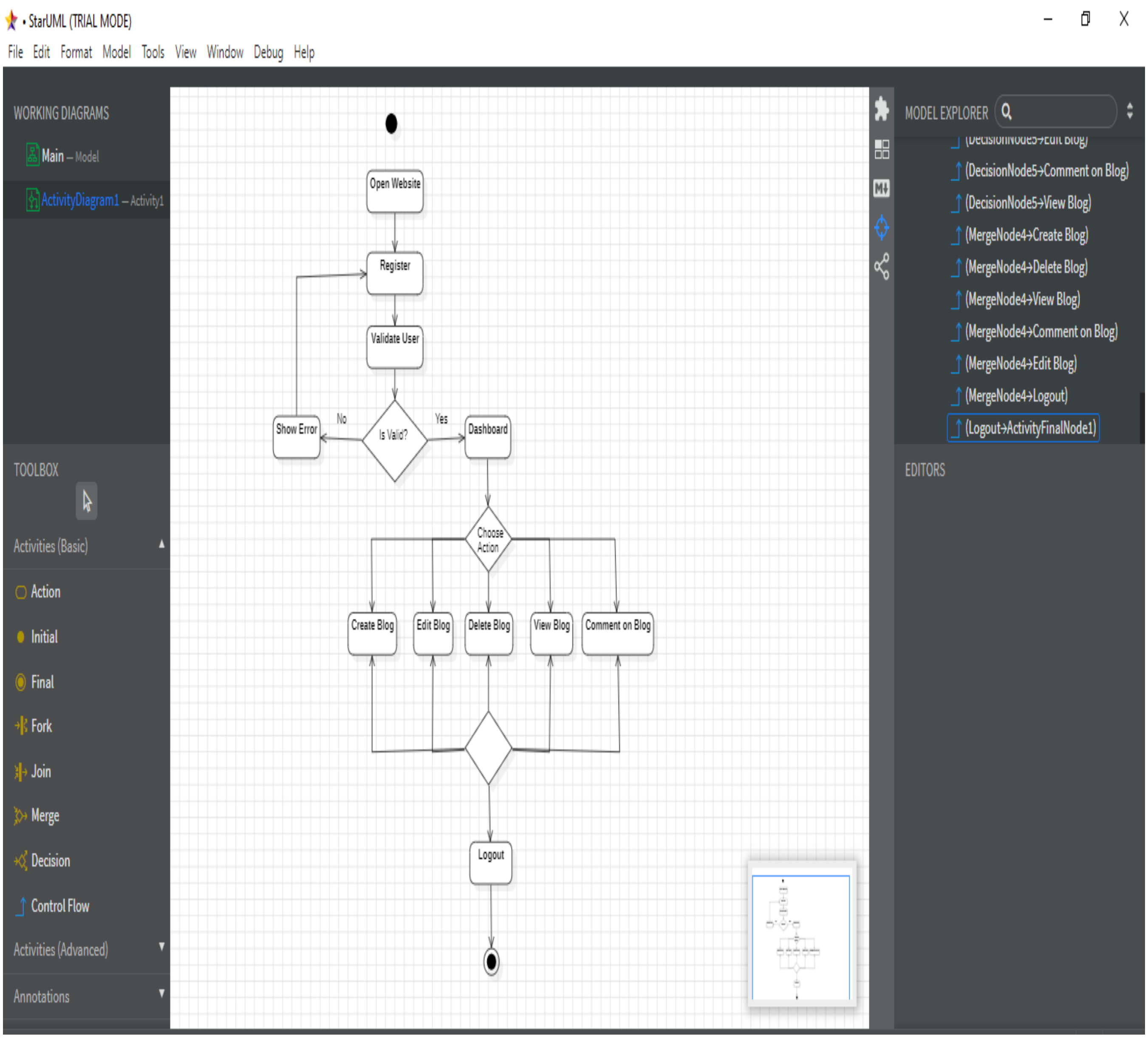
The Class Diagram and Sequence Diagram is designed Successfully for given project.

EXP NO: 8 **23/02/2026**	**DESIGNING USE CASE DIAGRAMS AND ACTIVITY DIAGRAM FOR PROJECT ARCHITECTURE**

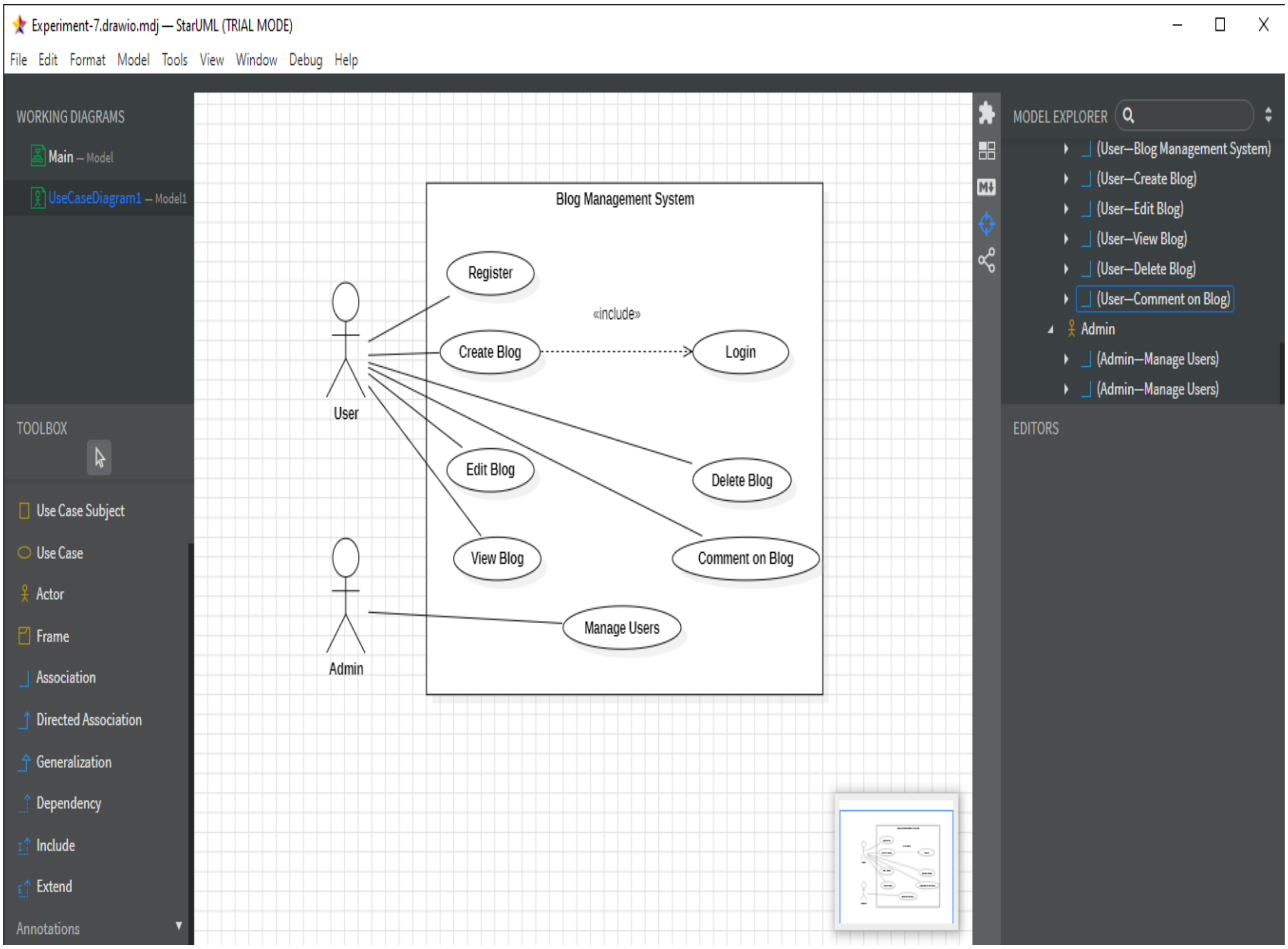
AIM:-

To Design an activity Diagram and use case Diagram for the given Project.

7A. Activity Diagram:-



7B. USE CASE DIAGRAM:-



RESULT:-

Thus activity and use case diagram has been designed successfully for Student Management System

EXP NO: 9 **09/03/2026**	**TESTING – TEST PLANS AND TEST CASES**

AIM:-

Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case
Test Case Design Procedure

Understand Core Features of the Application

- User Signup & Login
- Create, Edit, and Delete Blog Posts
- Commenting on Blog Posts
- Like/Share Functionality
- Viewing Blog Posts (Public/Private)
- Admin Role Management and Content Moderation

2. Define User Interactions

- Each test case simulates real user behavior (e.g., logging in, creating a post, commenting, liking a post, moderating content).

3. Design Happy Path Test Cases

- Focus on validating that all core functionalities work correctly under normal conditions.
- Example: User registers → logs in → creates a blog post → publishes it → other users view, like, and comment on the post.

4. Design Error Path Test Cases

- Simulate invalid inputs, system issues, or failed actions to ensure proper error handling.
- Examples:
  - Login with incorrect credentials
  - Submitting an empty blog post
  - Comment submission without text
  - Unauthorized user trying to edit/delete another user’s post
  - Accessing admin panel without permission

5. Break Down Steps and Expected Results

- Each test case includes a clear sequence of actions and expected results.
- Ensures both manual testers and automation tools can easily execute and validate outcomes.

6. Use Clear Naming and IDs

- Test cases are uniquely identified (e.g., TC01 – User Login Success, TC08 – Create Blog Post, TC15 – Invalid Comment Submission).
- Helps in tracking and mapping to features in tools like Azure DevOps.

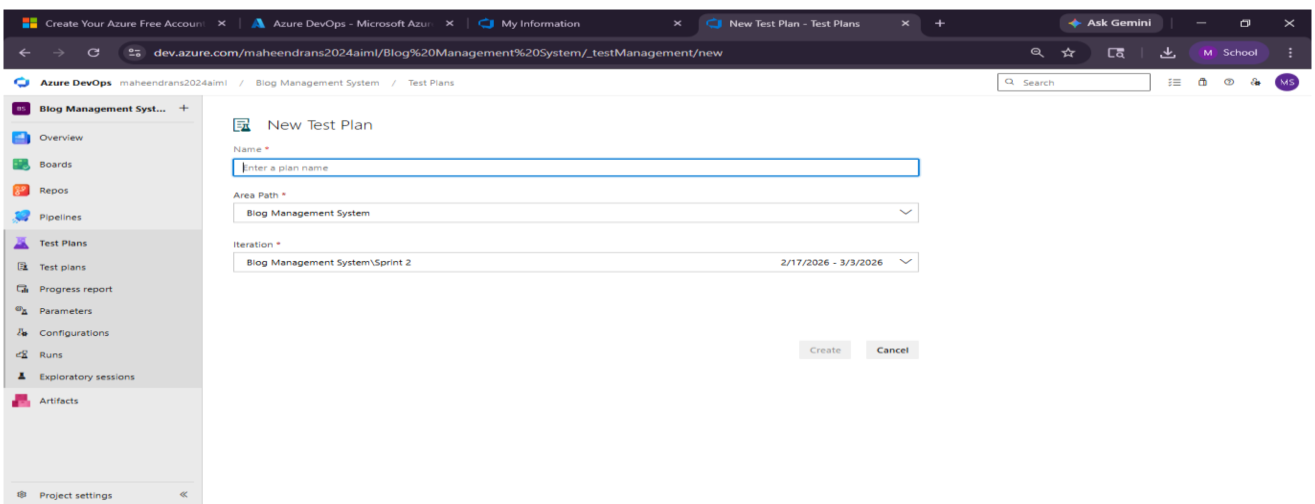
7. Separate Test Suites

- Group test cases by functionality such as:
    - Login and Registration
    - Blog Post Management
    - Comments and Interactions
    - Content Moderation
    - Admin Functions
- Improves organization and allows focused execution.

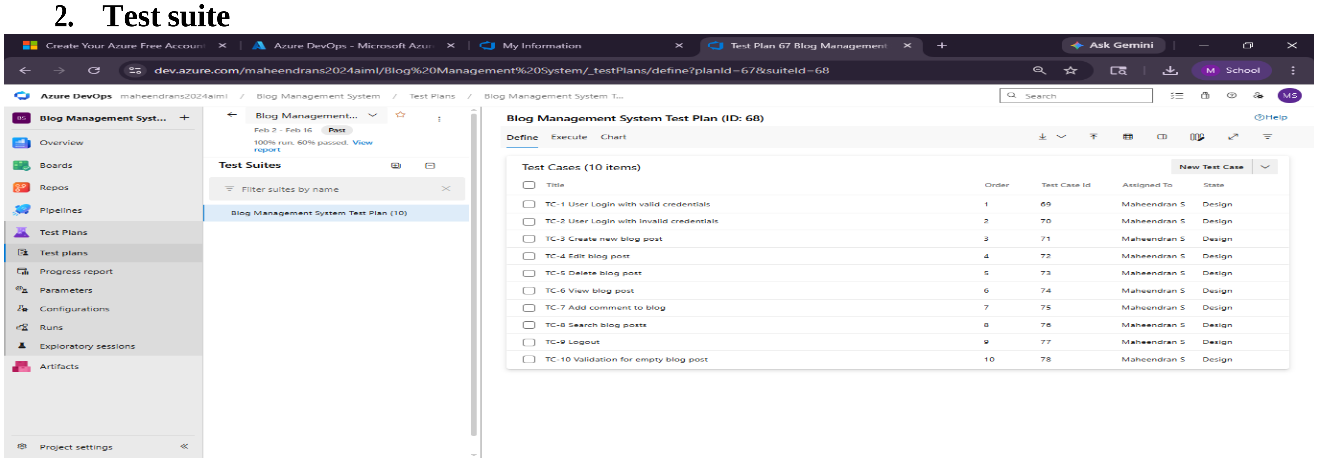
8. Prioritize and Review

- Assign high priority to critical workflows like login, post creation, publishing, and content moderation.
- Review for completeness, accuracy, and alignment with user stories and feature requirements.

1. New test plan



2. Test suite



3. Test case

Give two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Student Management System– Test Plans

USER STORIES

- **ID: 201** – As a user, I want to log in securely so I can access my blog dashboard
- **ID: 202** – As a user, I want to create blog posts so I can share content
- **ID: 203** – As a user, I want to edit blog posts so I can update content
- **ID: 204** – As a user, I want to delete blog posts so I can remove unwanted content
- **ID: 205** – As a user, I want to search blog posts to find specific content

Test Suite: TS01 – User Authentication

- **TC-1: User login with valid credentials**
  - User Story: ID 201
  - Type: Happy Path
- **TC-2: User login with invalid credentials**
  - User Story: ID 201
  - Type: Error Path

Test Suite: TS02 – Blog Creation

- **TC-3: Create new blog post with valid data**
  - User Story: ID 202
  - Type: Happy Path
- **TC-4: Create blog post with empty required fields**
  - User Story: ID 202
  - Type: Error Path

Test Suite: TS03 – Blog Editing

- **TC-5: Edit existing blog post successfully**
    - User Story: ID 203
    - Type: Happy Path
- **TC-6: Edit blog post with invalid or empty content**
    - User Story: ID 203
    - Type: Error Path

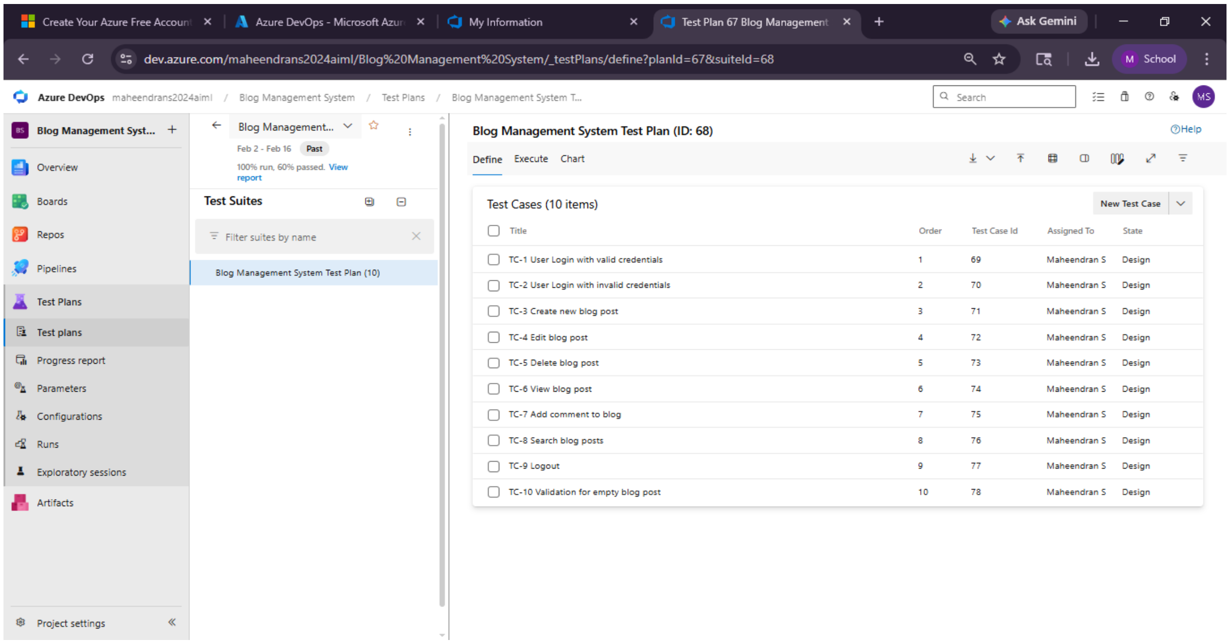
Test Suite: TS04 – Blog Deletion

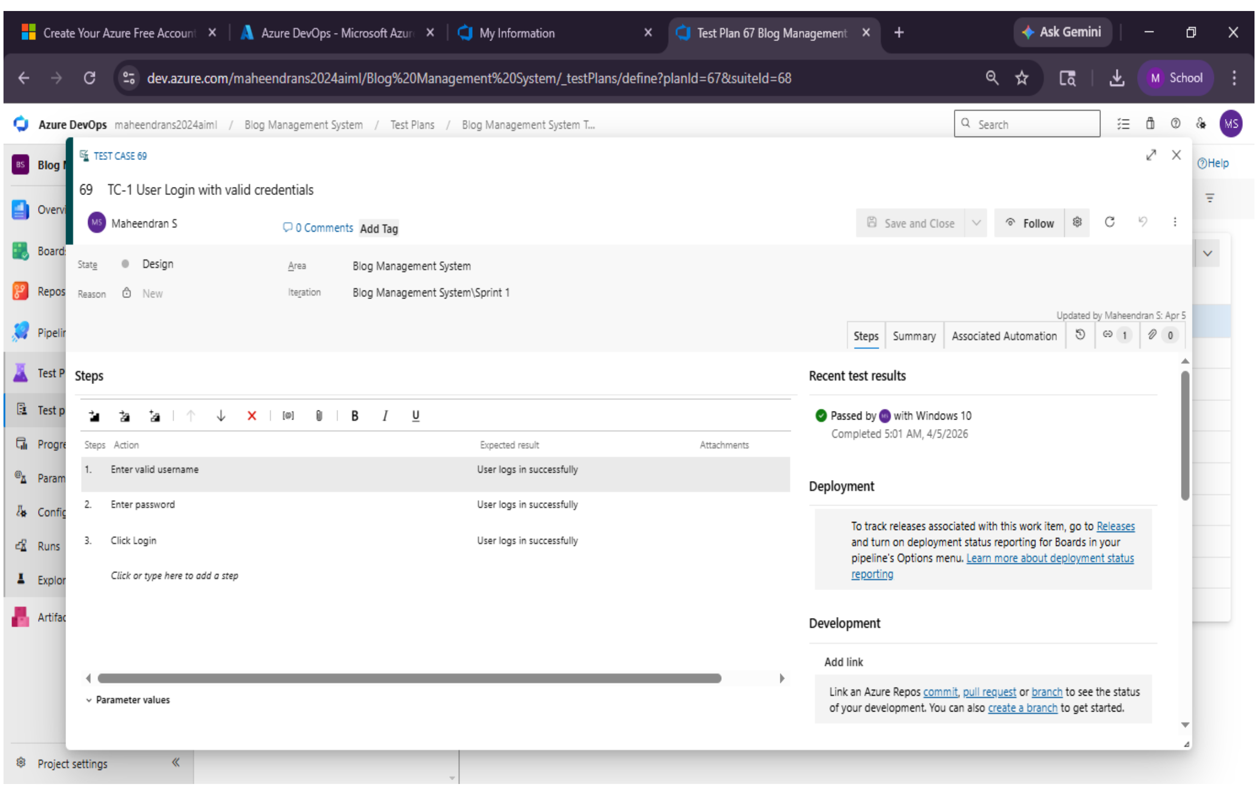
- **TC-7: Delete blog post successfully**
    - User Story: ID 204
    - Type: Happy Path
- **TC-8: Attempt to delete non-existing blog post**
    - User Story: ID 204
    - Type: Error Path

Test Suite: TS05 – Blog Search

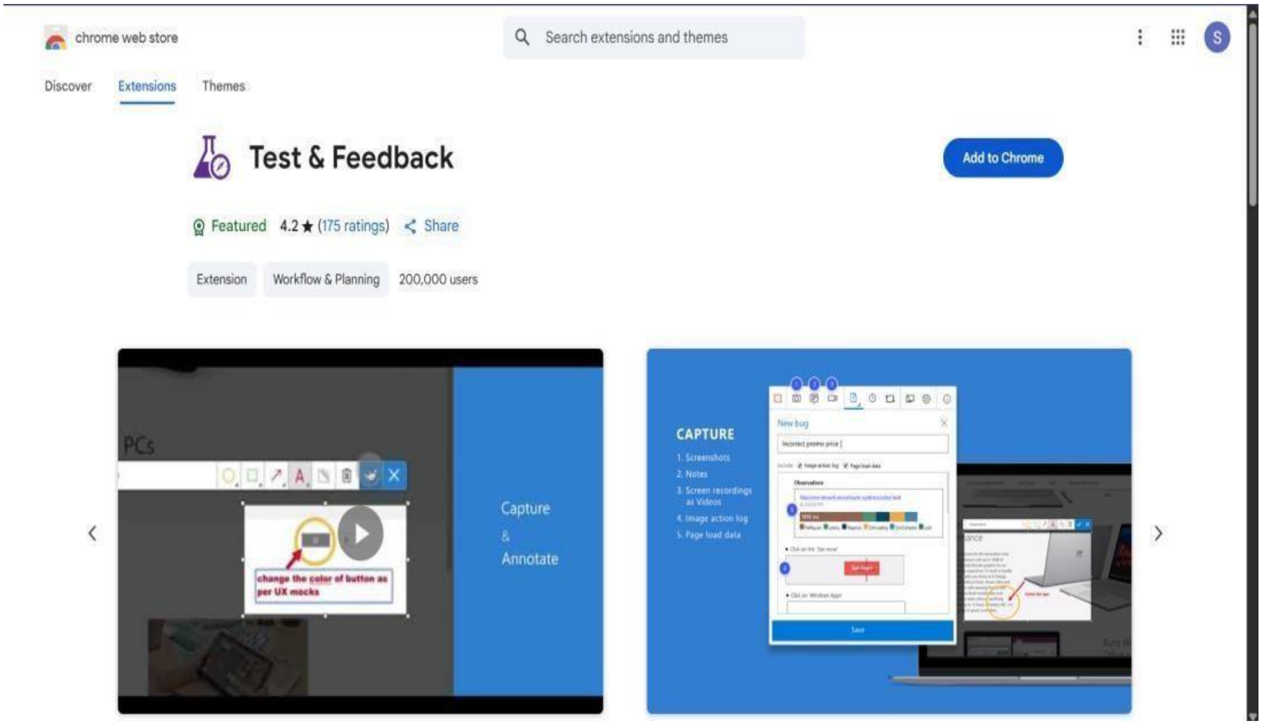
- **TC-9: Search blog posts with valid keyword**
    - User Story: ID 205
    - Type: Happy Path
- **TC-10: Search blog posts with no matching results**
    - User Story: ID 205
    - Type: Error Path

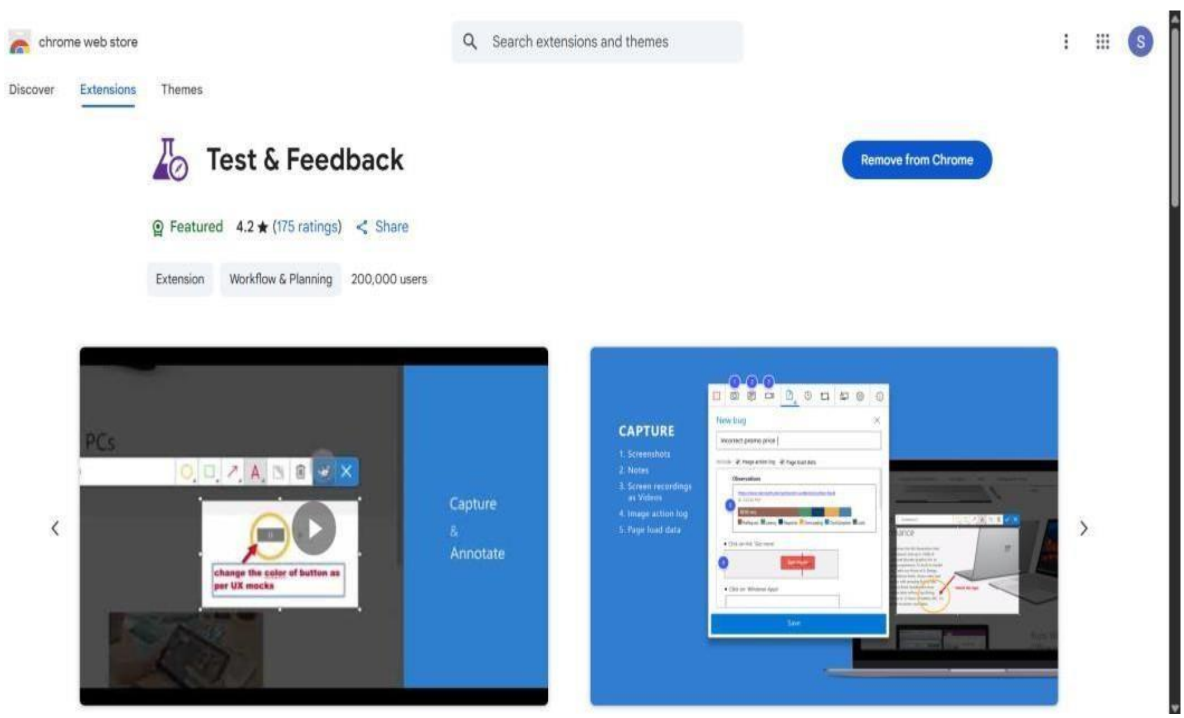
1. Test Cases


dev.azure.com/maheendrans2024aiml/Blog%20Management%20System/_testPlans/define?planId=67&suiteId=68
Azure DevOps
Blog Management System Test Plan (ID: 68)
Define
Execute
Chart
Test Suites
Blog Management System Test Plan (10)
Test Cases (10 items)
New Test Case
Title
Order
Test Case Id
Assigned To
State
TC-1 User Login with valid credentials 1 69 Maheendran S Design
TC-2 User Login with invalid credentials 2 70 Maheendran S Design
TC-3 Create new blog post 3 71 Maheendran S Design
TC-4 Edit blog post 4 72 Maheendran S Design
TC-5 Delete blog post 5 73 Maheendran S Design
TC-6 View blog post 6 74 Maheendran S Design
TC-7 Add comment to blog 7 75 Maheendran S Design
TC-8 Search blog posts 8 76 Maheendran S Design
TC-9 Logout 9 77 Maheendran S Design
TC-10 Validation for empty blog post 10 78 Maheendran S Design
Overview
Boards
Repos
Pipelines
Test Plans
Test plans
Progress report
Parameters
Configurations
Runs
Exploratory sessions
Artifacts
Project settings

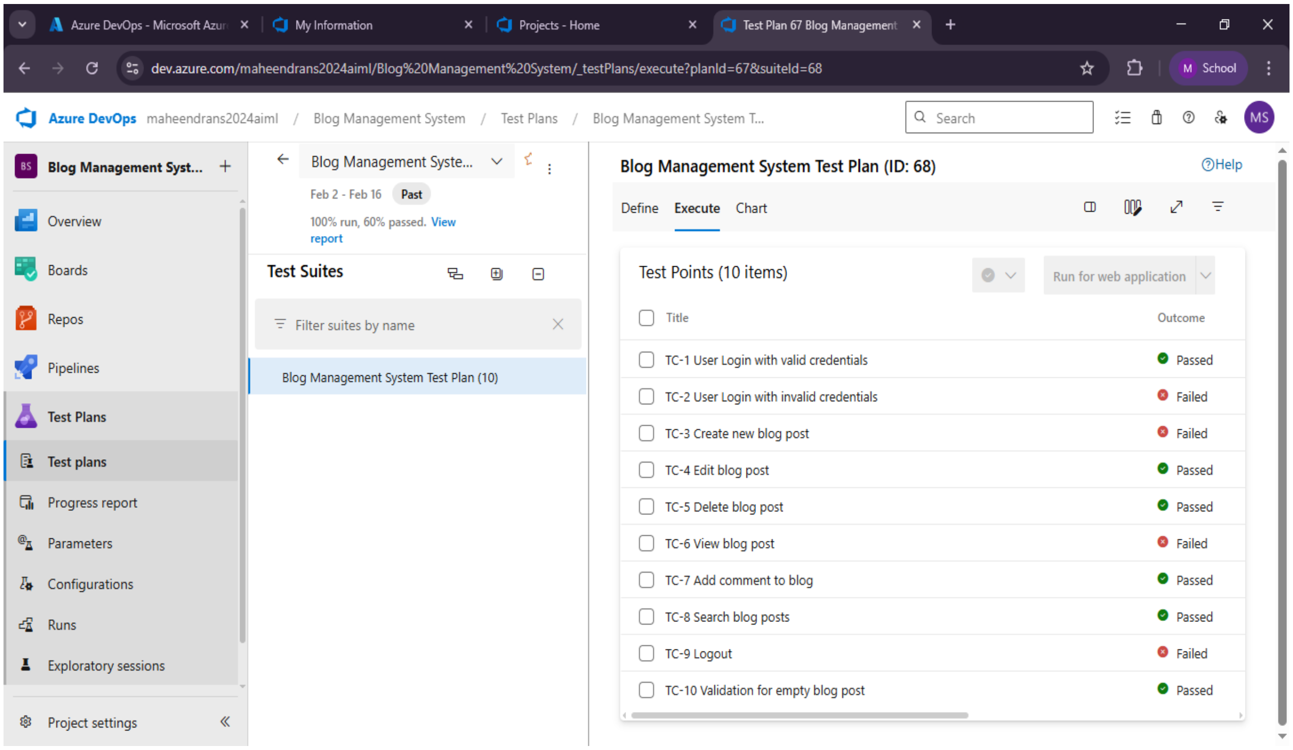

dev.azure.com/maheendrans2024aiml/Blog%20Management%20System/_testPlans/define?planId=67&suiteId=68
TEST CASE 69
69 TC-1 User Login with valid credentials
Maheendran S
0 Comments
Add Tag
Save and Close
Follow
State Design
Reason New
Area Blog Management System
Iteration Blog Management System\Sprint 1
Steps
Summary
Associated Automation
Steps
Action
Expected result
Attachments
1. Enter valid username User logs in successfully
2. Enter password User logs in successfully
3. Click Login User logs in successfully
Click or type here to add a step
Parameter values
Recent test results
Passed by with Windows 10
Completed 5:01 AM, 4/5/2026
Deployment
To track releases associated with this work item, go to Releases and turn on deployment status reporting for Boards in your pipeline's Options menu. Learn more about deployment status reporting
Development
Add link
Link an Azure Repos commit, pull request or branch to see the status of your development. You can also create a branch to get started.

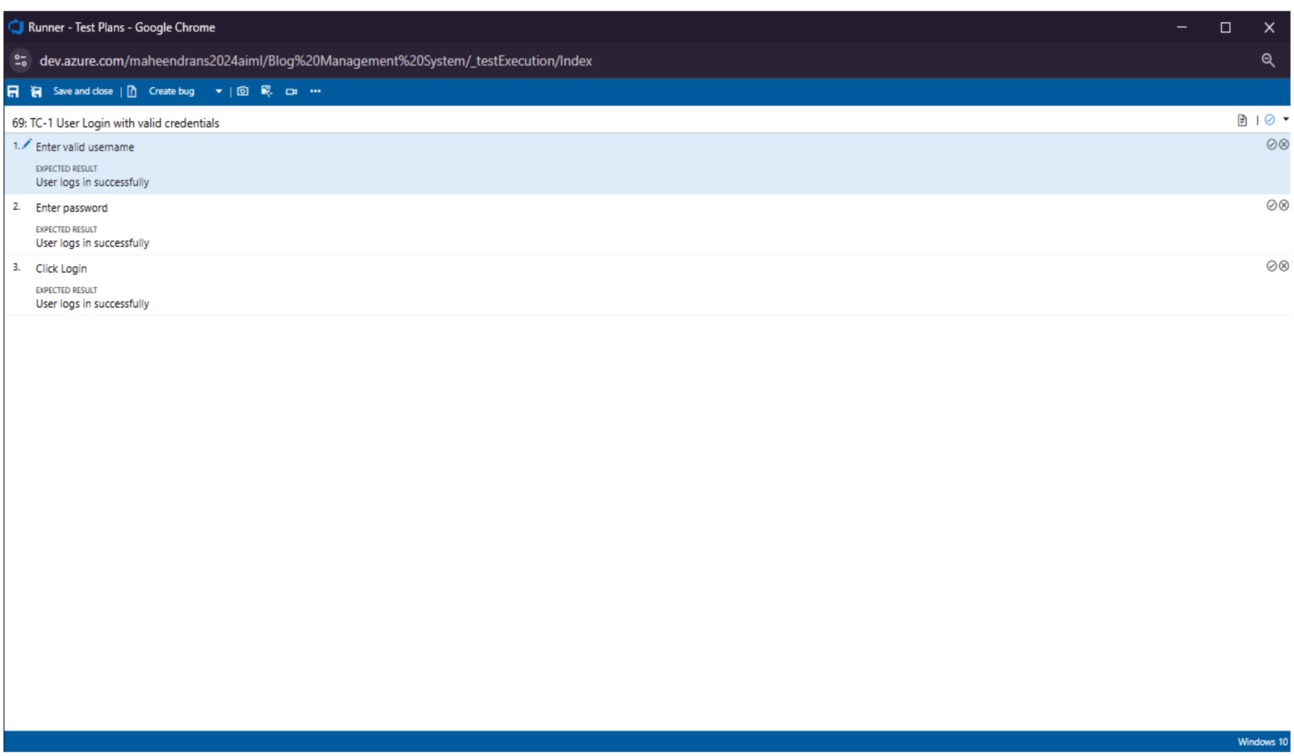
2. Installation of test



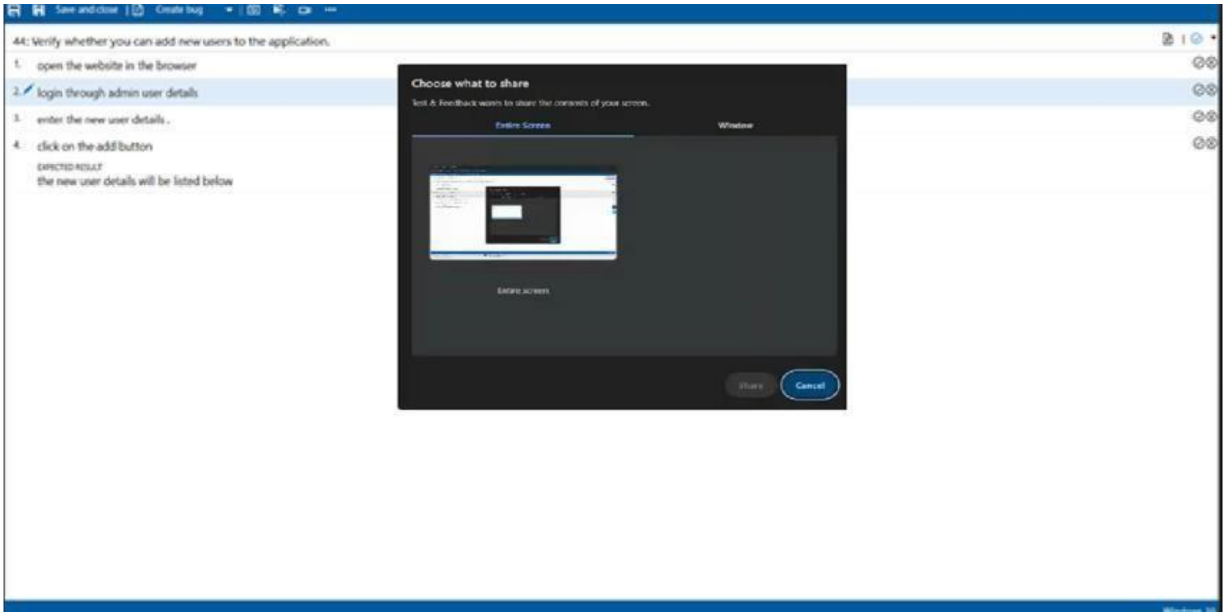


3. Running the test cases

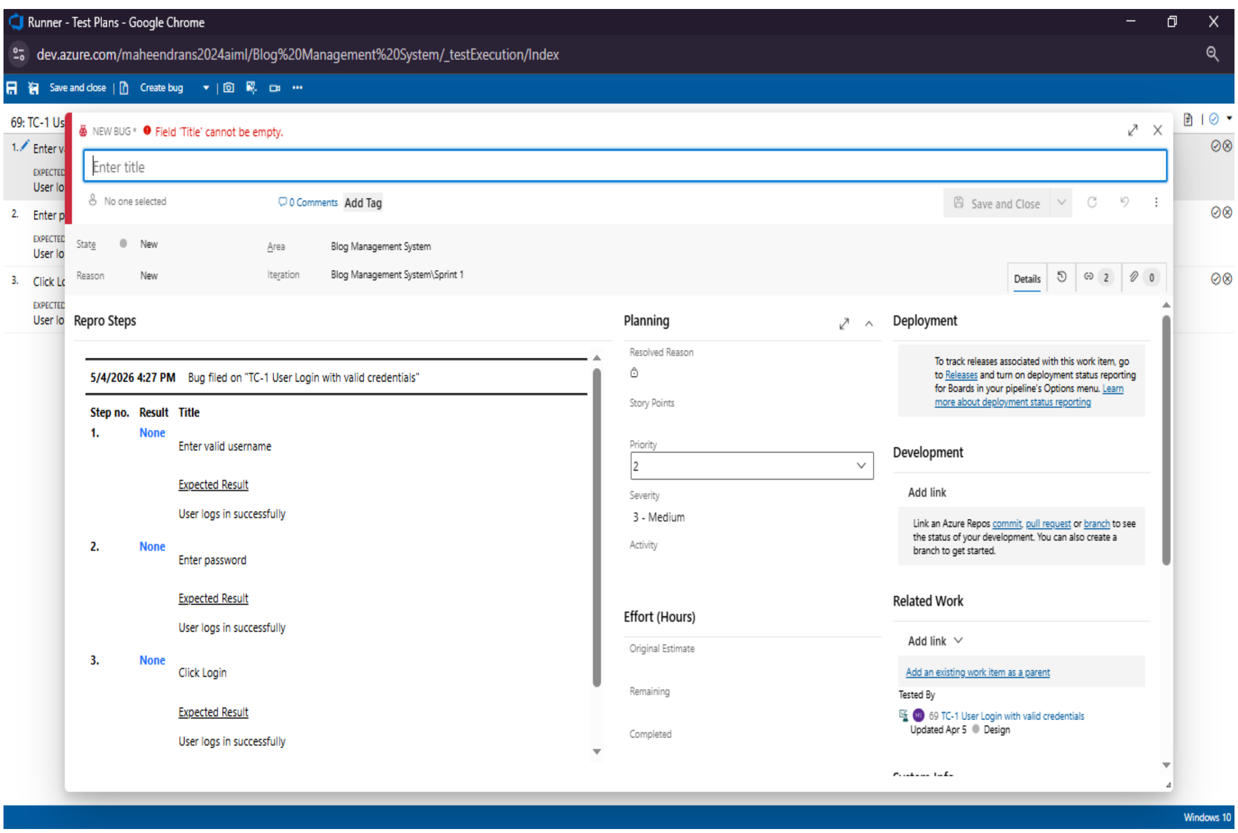


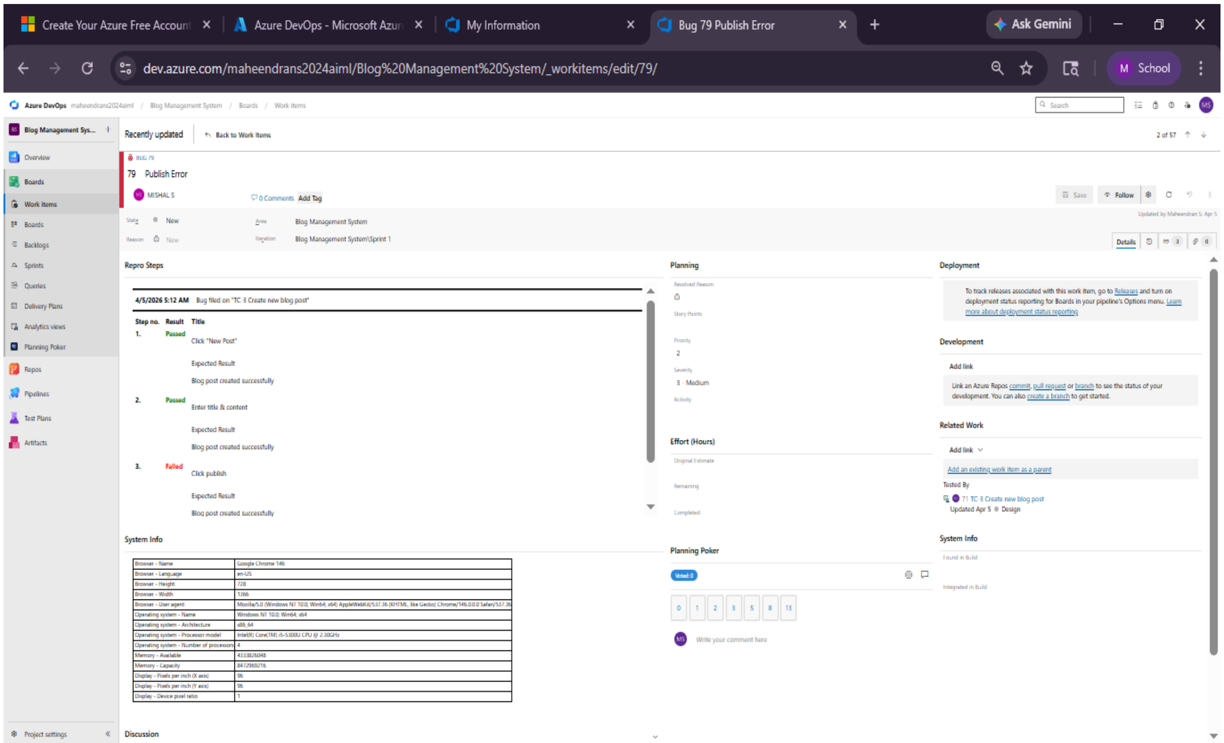


4. Recording the test case

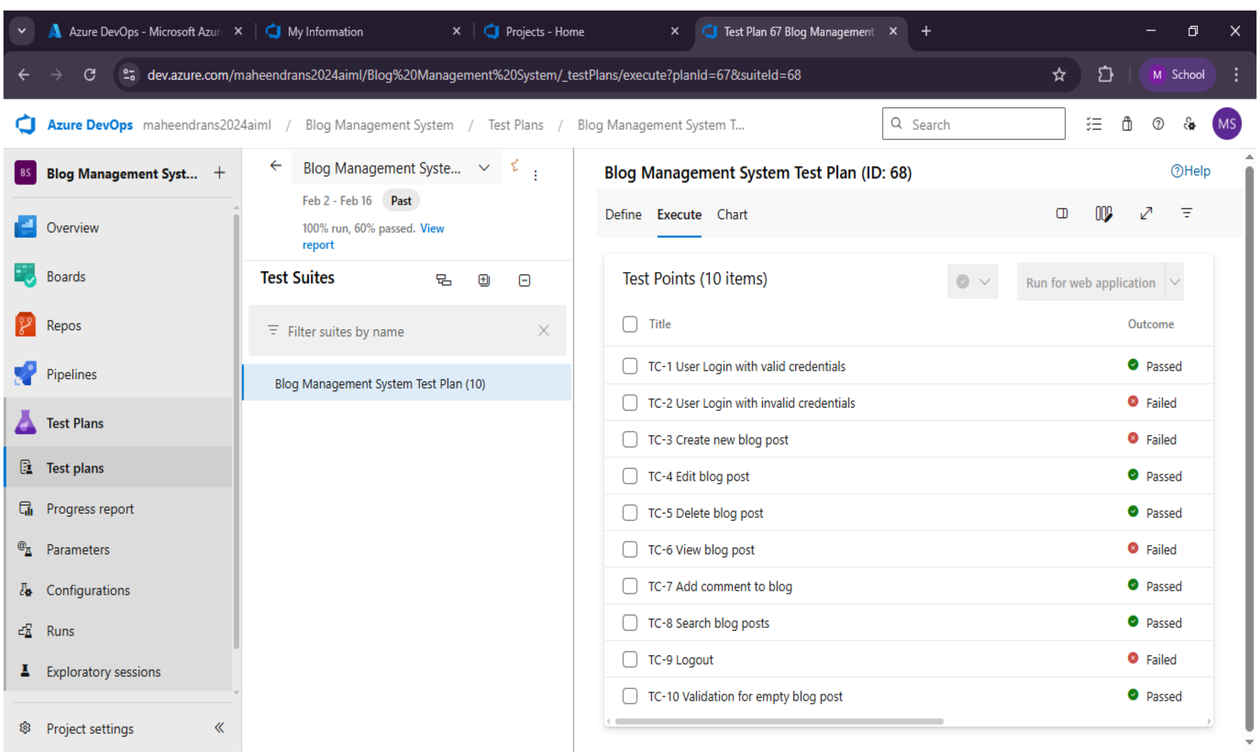


5. Creating the bug

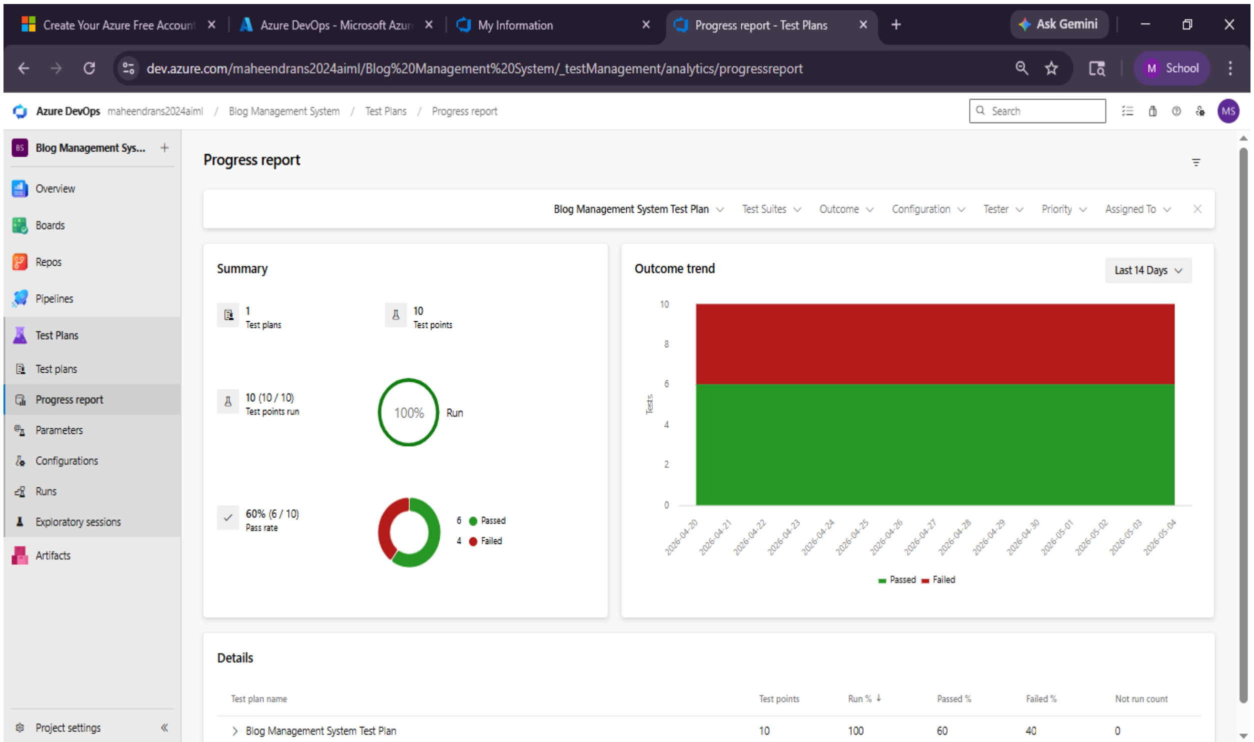




Test Results



6. Progress report



Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path

EXP NO : 10 16/03/2026	CREATION OF WEB PAGE

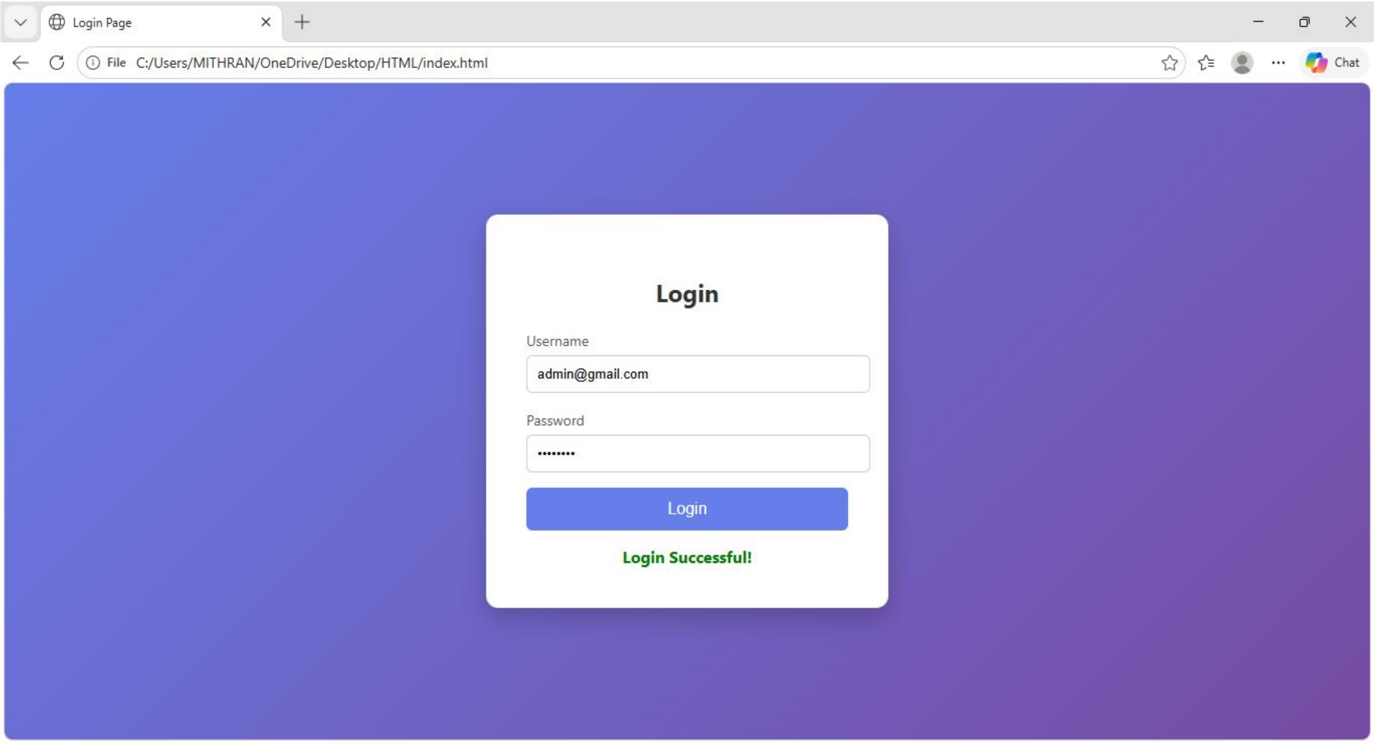
Aim

To design and create a basic web page using HTML (and optionally CSS) to display structured content such as text, images, and links in a web browser.

Algorithm

1. **Start**
2. Open a text editor (e.g., Notepad or VS Code)
3. Create a new file and save it with `.html` extension
4. Write the basic HTML structure:
   - `<html>`
   - `<head>` (title of the page)
   - `<body>` (content of the page)
5. Add elements such as headings, paragraphs, images, and links
6. Apply basic styling using CSS (optional)
7. Save the file
8. Open the file in a web browser
9. Verify the output and make necessary changes
10. **End**

OUTPUT:



Result

The web page is successfully created and displayed in a web browser with the required content, structure, and basic styling.

EXP NO: 11 **06/04/2026**	**CI/CD PIPELINES IN AZURE**

AIM:

To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline in Azure DevOps for automating the build, testing, and deployment process of the Student Management System, ensuring faster delivery and improved software quality.

PROCEDURE:

Steps to Create and implement pipelines in Azure:

1. Sign in to Azure DevOps and Navigate to Your Project
   Log in to dev.azure.com, select your organization, and open the project where your Student Management System code resides.

2. Connect a Code Repository (Azure Repos or GitHub)
   Ensure your application code is stored in a Git-based repository such as Azure Repos or GitHub. This will be the source for triggering builds and deployments in your pipeline.

3. Create a New Pipeline
   Go to the Pipelines section on the left panel and click "Create Pipeline".
   Choose your source (e.g., Azure Repos Git or GitHub), and then select the repository containing your project code.

4. Choose the Pipeline Configuration
   You can select either the YAML-based pipeline (recommended for version control and automation) or the Classic Editor for a GUI-based setup.
   If using YAML, Azure DevOps will suggest a template or allow you to define your own.

5. Define Build Stage (CI - Continuous Integration) from YAML file

6. Install dependencies (e.g., npm install, dotnet restore)

7. Build the application (dotnet build, npm run build)

8. Run unit tests (dotnet test, npm test)

9. Publish build artifacts to be used in the release stage

10. Save and Run the Pipeline for the First Time
    Save the YAML or build definition and click “Run”.
    Azure will fetch the latest code and execute the defined build and test stages.

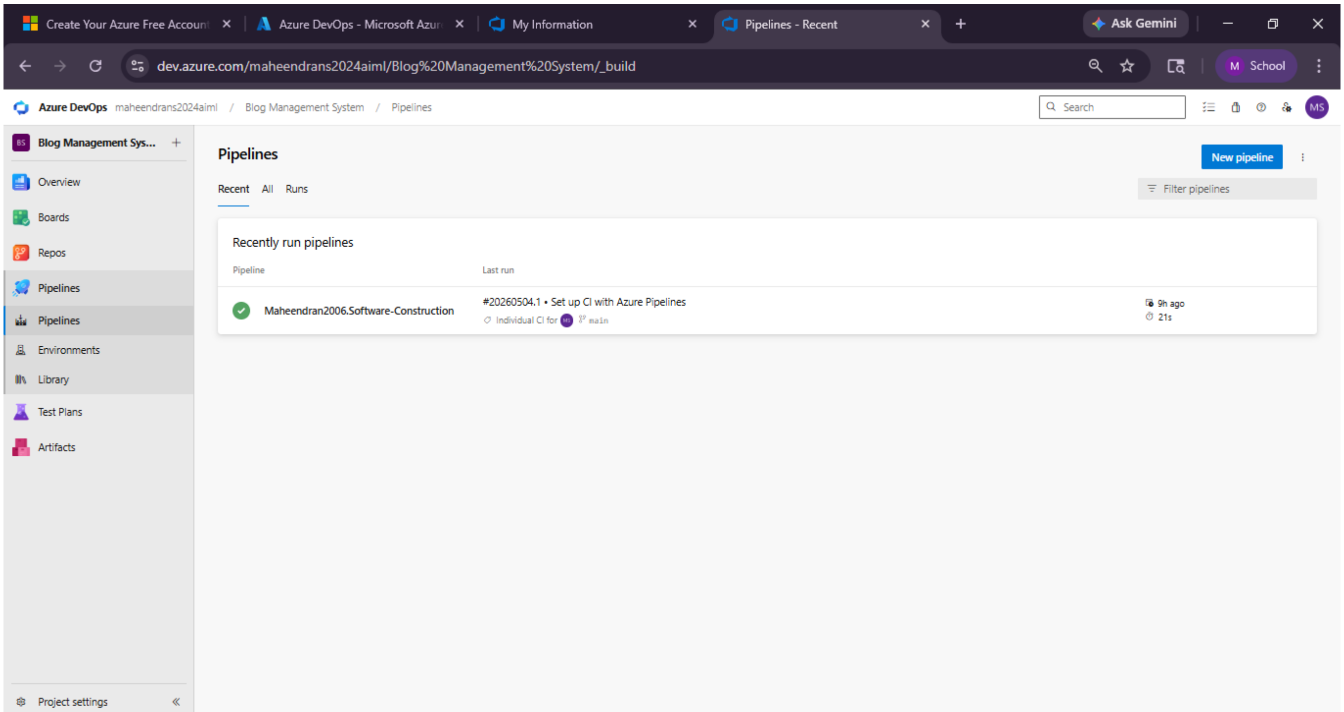
11. Configure Continuous Deployment (CD)
    Navigate to the Releases tab under Pipelines and click “New Release Pipeline”.
    Add an Artifact (from the build stage) and create a new Stage (e.g., Development, Production).

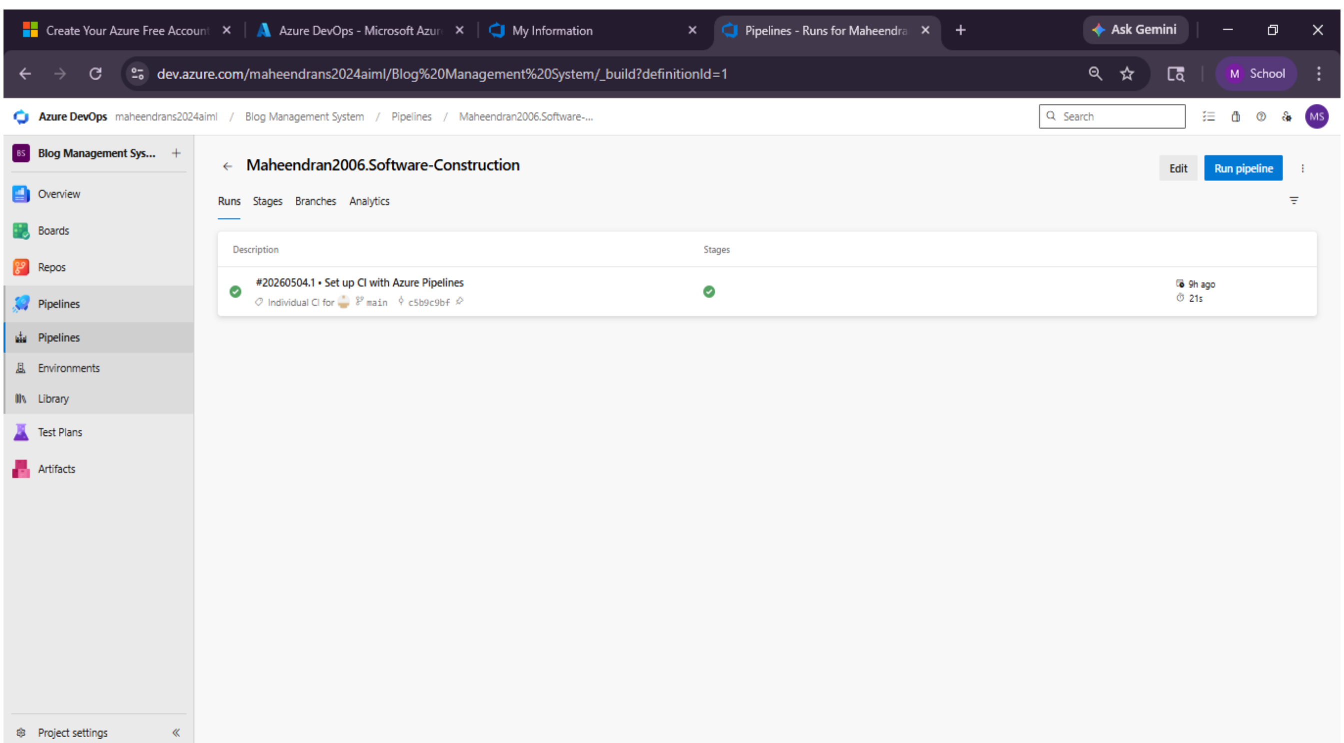
12. Configure the CD stage with deployment tasks such as deploying to Azure App Service, running database migrations or scripts, and restarting services using the Azure App Service Deploy task linked to your subscription and app details.

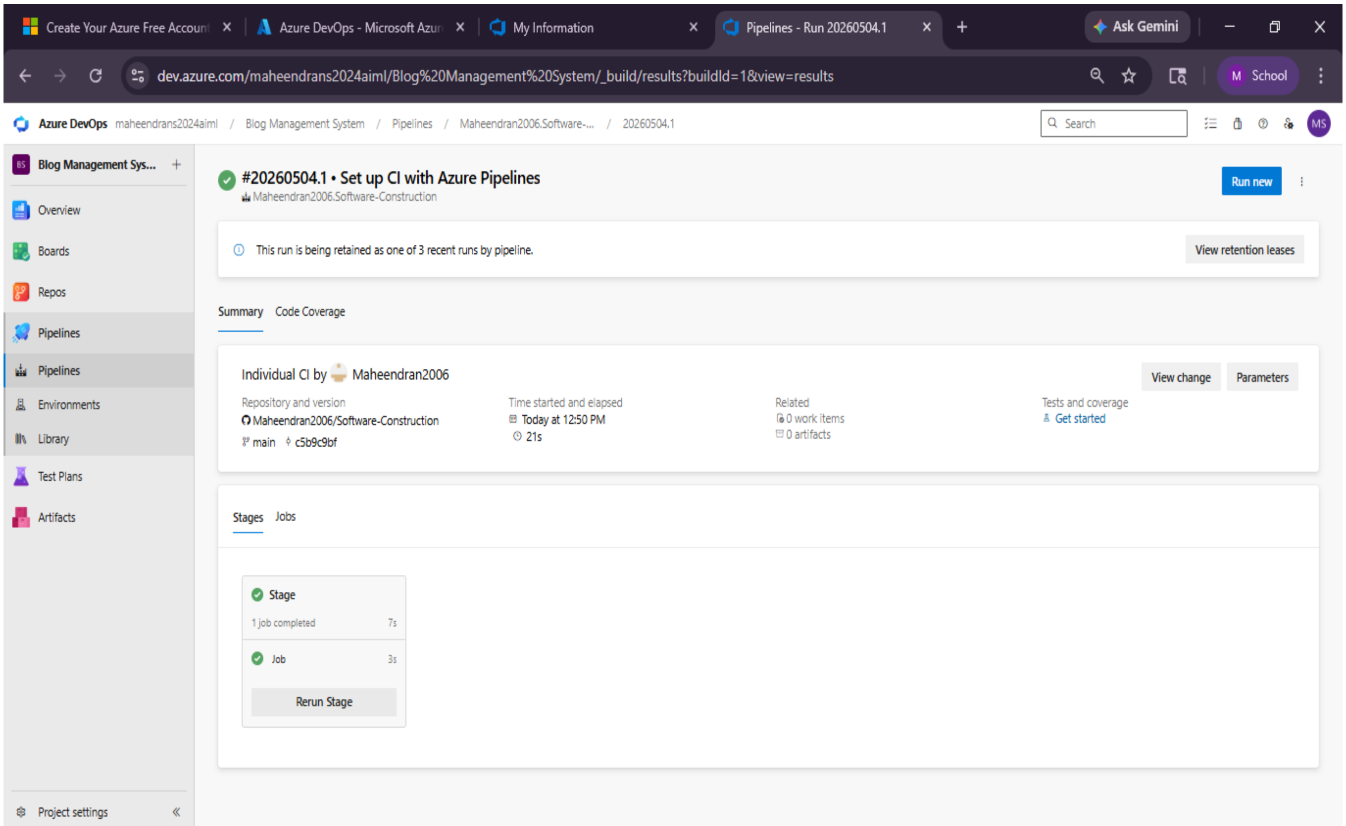
13. Set Triggers and Approvals
    Enable continuous deployment trigger so the release pipeline runs automatically after a successful build.
    For production environments, configure pre-deployment approvals to ensure manual verification before release.

14. Monitor Pipelines and Manage Logs
    View all pipeline runs under the Runs section.
    Check logs for build/test/deploy stages to debug any errors.
    You can also integrate email alerts or Microsoft Teams notifications for build failures.

15. Review and Maintain Pipelines
    Regularly update your pipeline tasks or YAML configurations as your application grows.
    Ensure pipeline runs are clean and artifacts are stored securely.
    Integrate quality gates and code coverage policies to maintain code quality.


Pipelines
Recently run pipelines
Maheendran2006.Software-Construction
#20260504.1 • Set up CI with Azure Pipelines


Maheendran2006.Software-Construction
#20260504.1 • Set up CI with Azure Pipelines



RESULT:

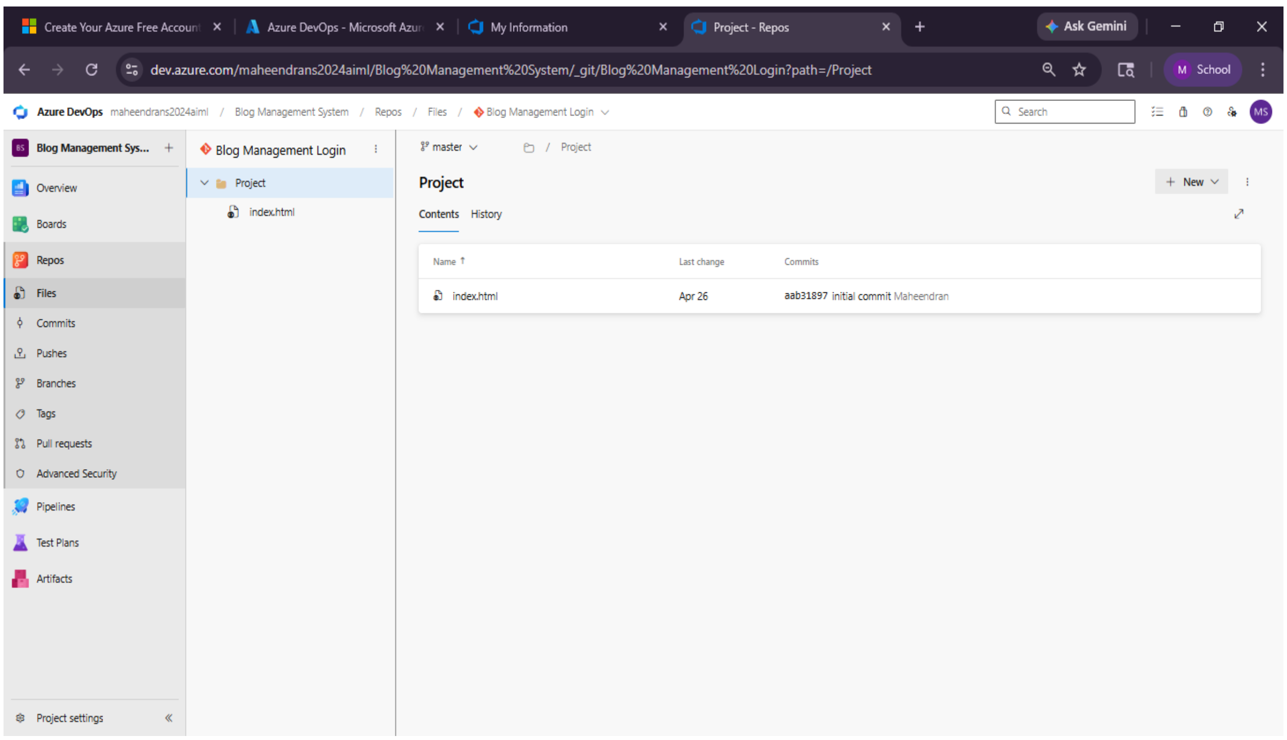
Thus the pipelines for the given project “Blog Management System has been executed successfully

EXP NO: 12 20/04/2026	GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Blog Management System project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.